

Visión Artificial

Tema 4. Extracción de características

PEDRO ENRIQUE LÓPEZ DE TERUEL ALCOLEA
DEPARTAMENTO INGENIERÍA Y TECNOLOGÍA DE COMPUTADORES
FACULTAD DE INFORMÁTICA

Universidad de Murcia

Máster en Inteligencia Artificial

2025/2026





- ▶ Imagen = array matricial de valores de gris/color → poco descriptivo.
- ▶ Buscamos una representación más **informativa** o **significativa**. Adecuada, p.e, para:
 - *Matching* entre imágenes (continuo, o “*wide baseline*”):
 - Profundidad a partir de dos (estéreo) o más vistas (reconstrucción 3D).
 - *Image retrieval* (obtener imágenes que contienen un modelo).
 - Localización (p.e. de un robot móvil en un determinado entorno).
 - Calibrar cámaras (*matching* con plantillas de calibración).
 - Panoramas.
 - Etc...
 - Detección y/o clasificación (caras, personas, objetos, etc.)

Extracción de características

Introducción a las características de imagen (*features*)



Máster en
Inteligencia
Artificial
UMU

- Ejemplo de extracción de *features*: puntos y líneas



Extracción de características

Introducción a las características de imagen (*features*)



► Ejemplo de matching entre *features*:



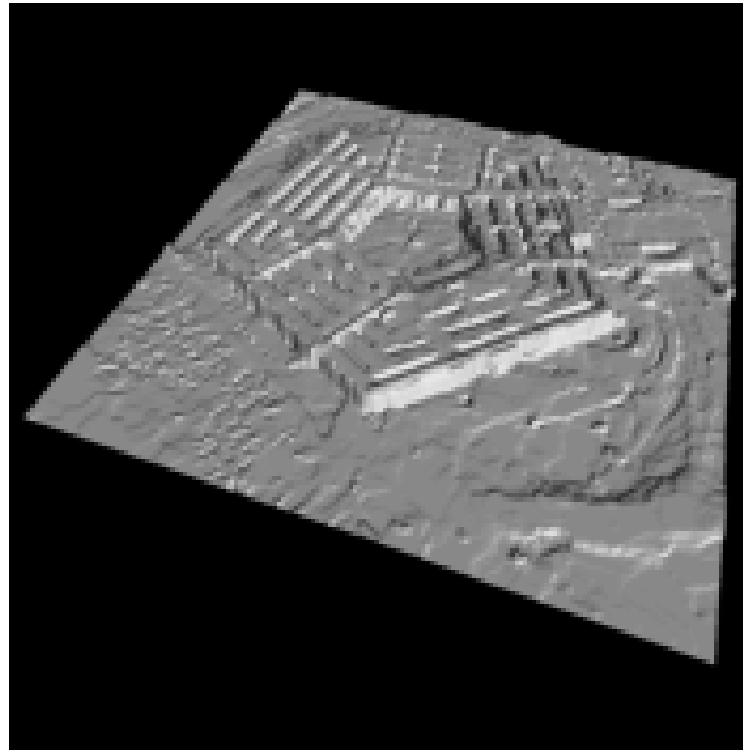
Extracción de características

Ejemplo de reconstrucción 3D



► Resultado:

- Usando técnicas de geometría proyectiva, y las restricciones de posición de cámaras y escena impuestas por el *matching* anterior → alineamiento de imágenes (estilo NB1) → *stereo* denso (mapa de profundidad):



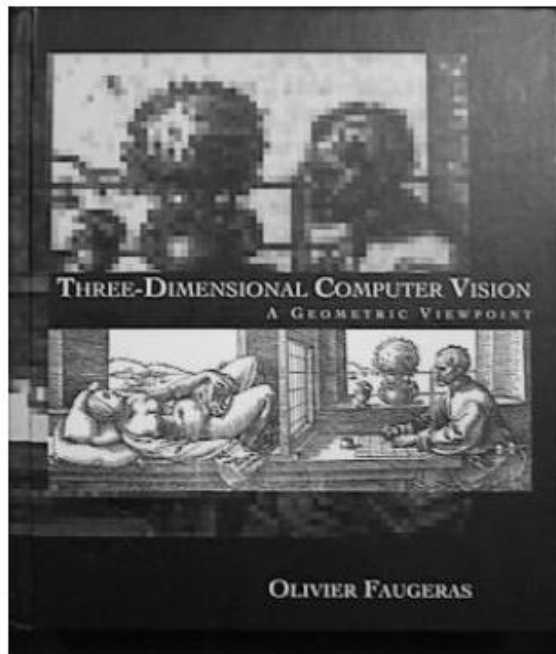
Extracción de características

Ejemplo de reconocimiento de objetos



Máster en
Inteligencia
Artificial
UMU

Queremos
Encontrar esto...



...aquí



Extracción de características

Ejemplo de construcción de panoramas



- Imágenes de partida: dos o más con solapamiento aparente:

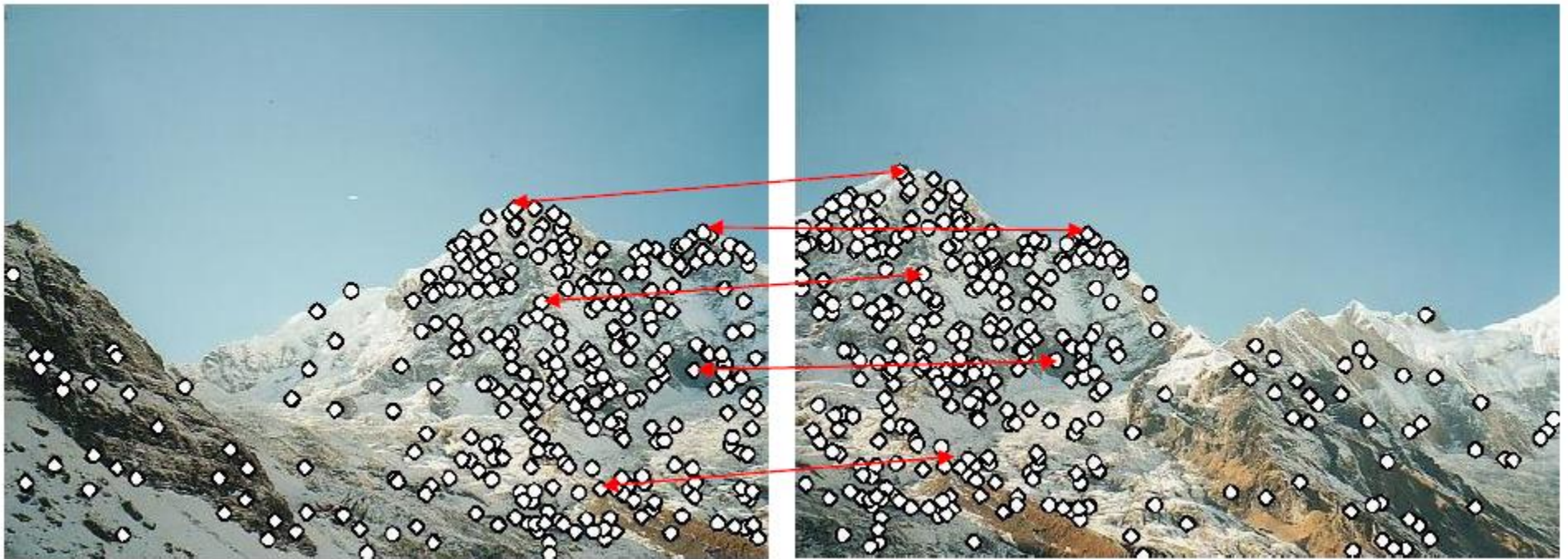


Extracción de características

Ejemplo de construcción de panoramas



► Extracción de características, y *matching*:



Extracción de características

Ejemplo de construcción de panoramas



Máster en
Inteligencia
Artificial
UMU

- *Image stitching*: Basado en estimación de la homografía interimagen, usada para el alineamiento.



Posteriormente
habría que
“trabajar”
el *blending*...

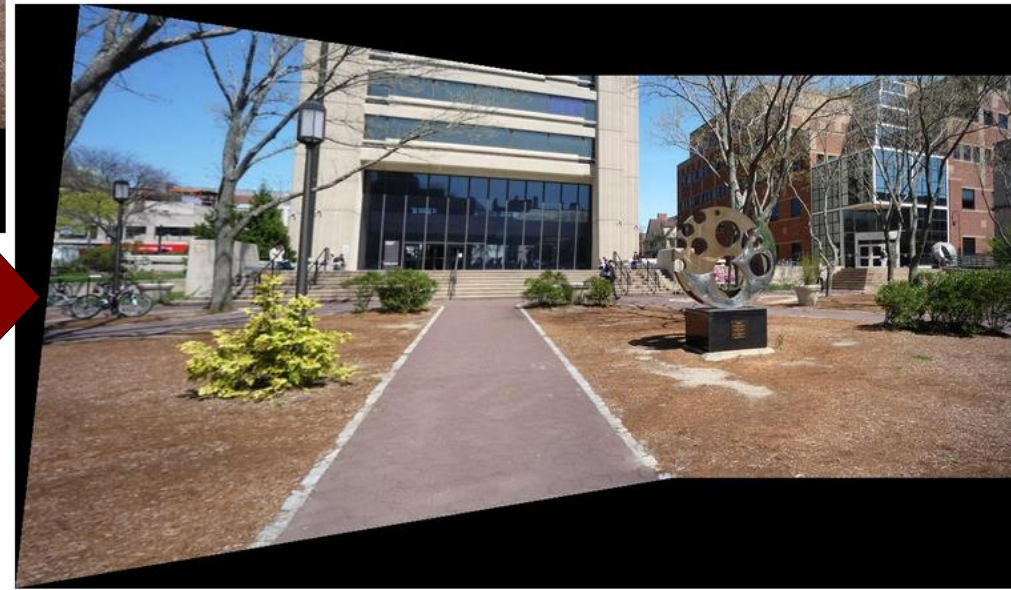
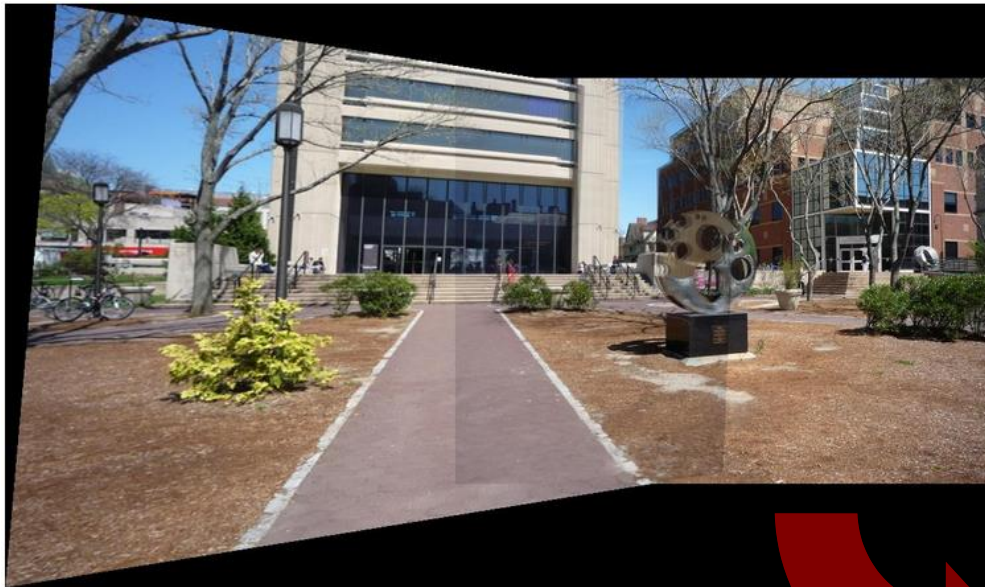
Extracción de características

Ejemplo de construcción de panoramas (II)



Máster en
Inteligencia
Artificial
UMU

► Ejemplo de *seamless blending*:



Extracción de características

Métodos de extracción de características **manuales**



- ▶ Métodos para intentar hacer decisiones más o menos locales en cada punto de la imagen, para decidir si en dicho punto existe una *feature* de un tipo dado o no.
 - Los métodos de extracción se consideran procesamiento de “bajo nivel”.
 - Por supuesto, bastante distintos según el tipo de característica a buscar.
- ▶ Tipos de *features*:
 - **Puntos** aislados, **bordes**, **regiones** conectadas, **segmentos**...
 - Definición ciertamente laxa, dependiente de la aplicación.
 - Por supuesto, luego a cada *feature* se asociarán una serie de propiedades (***descriptores***), que serán utilizadas para hacer ***matching***.

Extracción de características

Múltiples tipos de *features*



Máster en
Inteligencia
Artificial
UMU

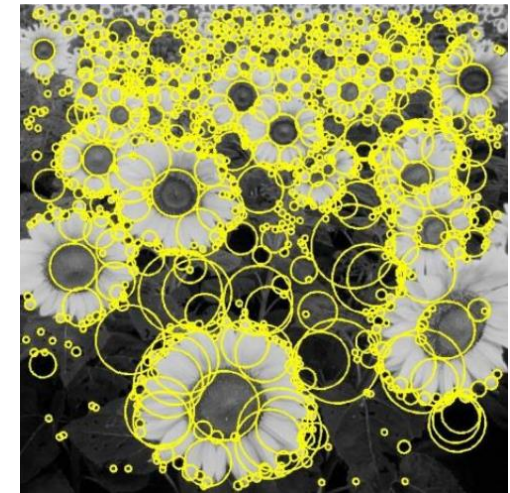
- Puntos con orientación local (a), regiones (b), bordes (c), segmentos (d), *blobs* (fundamentales para SIFT) (e), puntos sin escala (f), ...



(a)



(b)



(c)



(c)



(d)



(f)

► Repetibilidad:

- Las características deben ser detectadas en ambas imágenes (¡en otro caso, no hay posibilidad de *matching*!):



► Discernibilidad:

- Necesidad de un buen descriptor, para guiar el *matching*:





- ▶ **Estabilidad** en la posición en distintas imágenes (precisión en la localización).
- ▶ Buscar **invarianza a transformaciones** euclídeas / afines / perspectiva (en la medida de lo posible)
- ▶ Buscar **invarianza a cambios de iluminación.**
- ▶ Y por supuesto... ¡**eficiencia!**

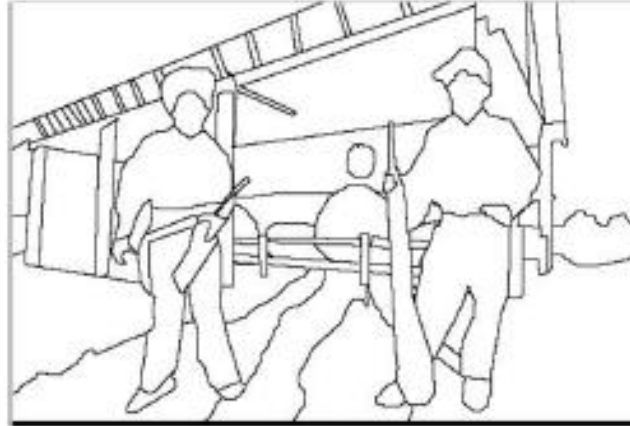
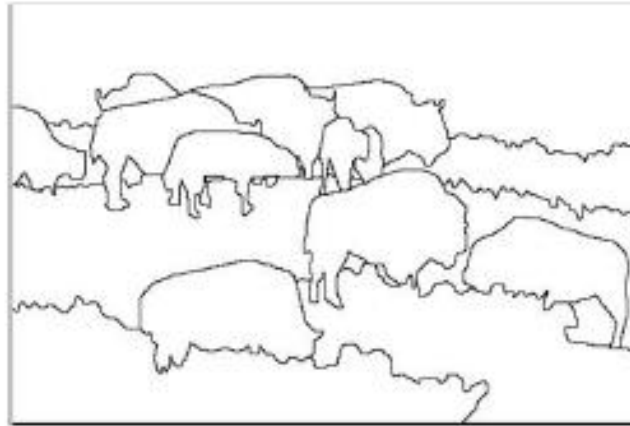
Extracción de características

Métodos de extracción de bordes



Máster en
Inteligencia
Artificial
UMU

- La percepción natural de los bordes está muy relacionada con los cambios locales de intensidad (gradiente):



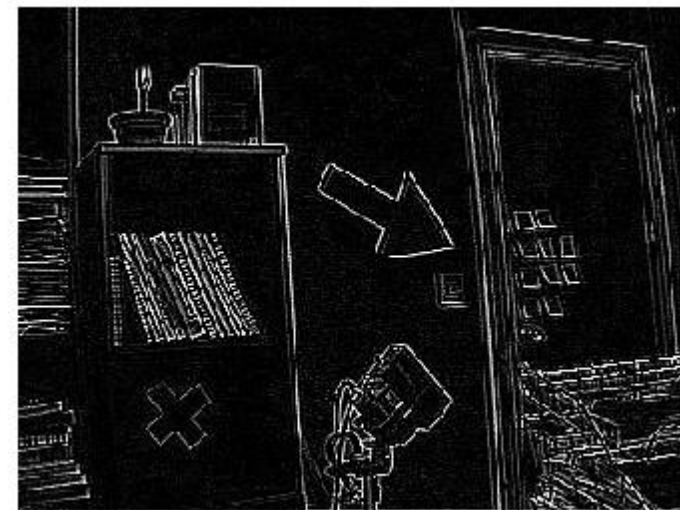
Original

Bordes etiquetados
por humanos

Módulo del
gradiente

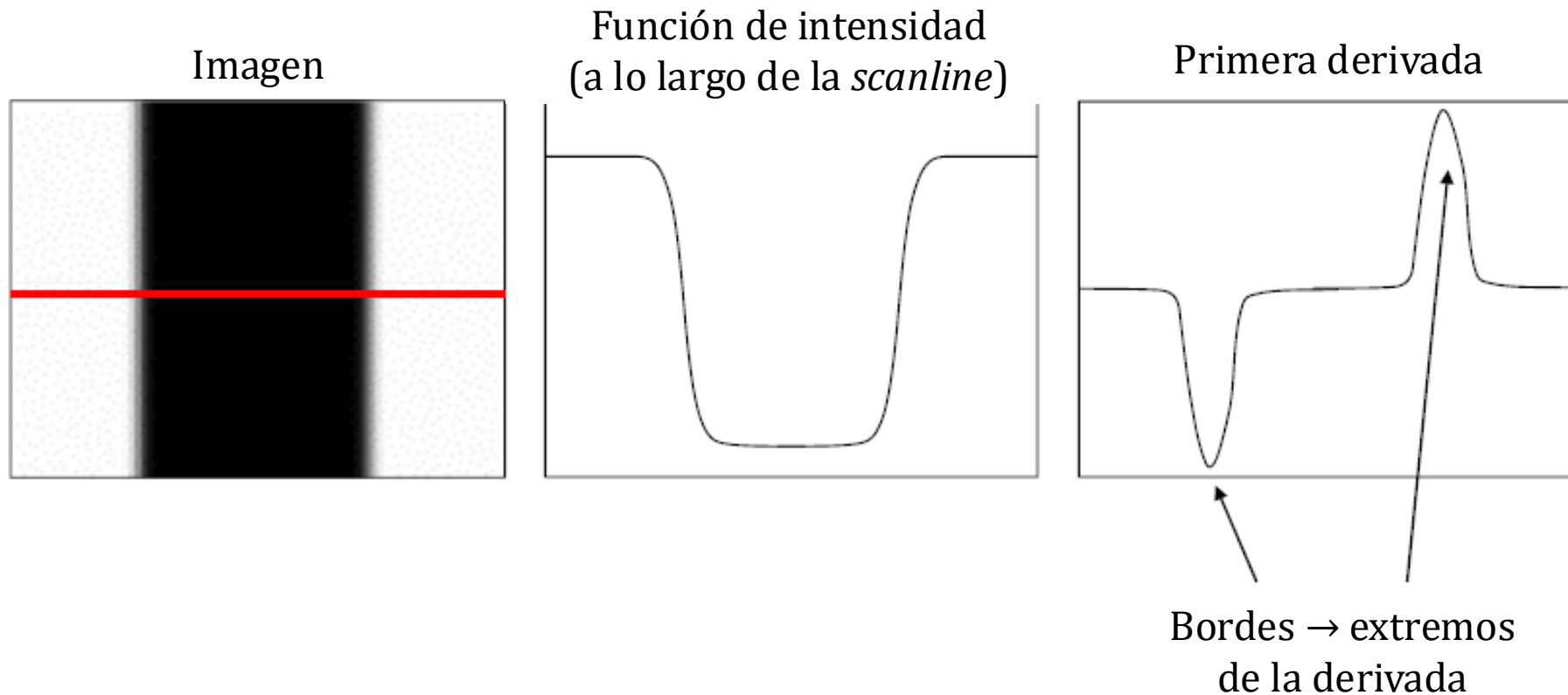
- ▶ Aproximación “ingenua”: operador de borde local lineal + umbralizado
- ▶ Ejemplo: filtro *highpass* (pasa-alta, deja pasar las “altas frecuencias”):

-1	-1	-1
-1	8	-1
-1	-1	-1



► Caracterización de bordes (*edges*):

- Lugar de cambio rápido en la variación de la función de intensidad:



► Máscaras lineales más utilizadas para la estimación de las derivadas direccionales en X e Y:

- Roberts (simple diferencia finita en X e Y, o bien operador cruzado):

$$\begin{bmatrix} -1 & +1 \end{bmatrix} \quad \begin{bmatrix} -1 \\ +1 \end{bmatrix} \quad \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix} \quad \begin{bmatrix} 0 & +1 \\ -1 & 0 \end{bmatrix}$$

- Prewitt (toma en cuenta entorno algo mayor):

$$\begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix}$$

- Sobel (mejor, ya que en realidad realiza también el equivalente a un presuavizado):

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

- Derivadas direccionales en X e Y (se computan con sencillos operadores lineales, como Roberts, Sobel, Prewitt):

Imagen vectorial $\longrightarrow$ $G(x) = \nabla I(x) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right)(x).$

- A partir de ellas, estimación de magnitud y orientación:

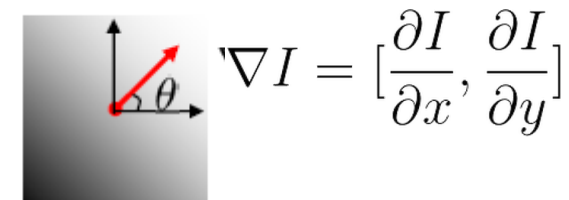
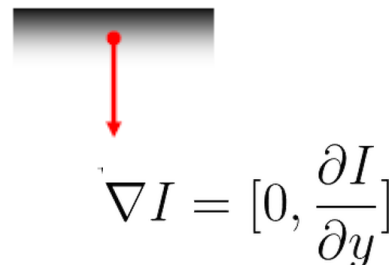
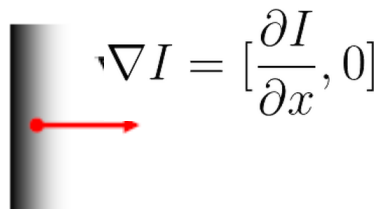
$$G_x = \partial I / \partial x$$

$$G_y = \partial I / \partial y$$

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\Theta = \text{atan2}(G_y, G_x)$$

- El gradiente es un vector que apunta en la dirección de máxima variación de la imagen, y es perpendicular a los cont



Extracción de características

Estimación local del gradiente



Máster en
Inteligencia
Artificial
UMU

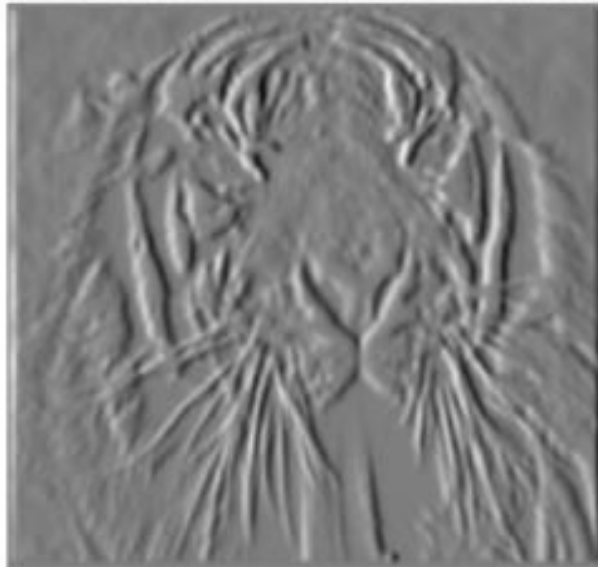
Original



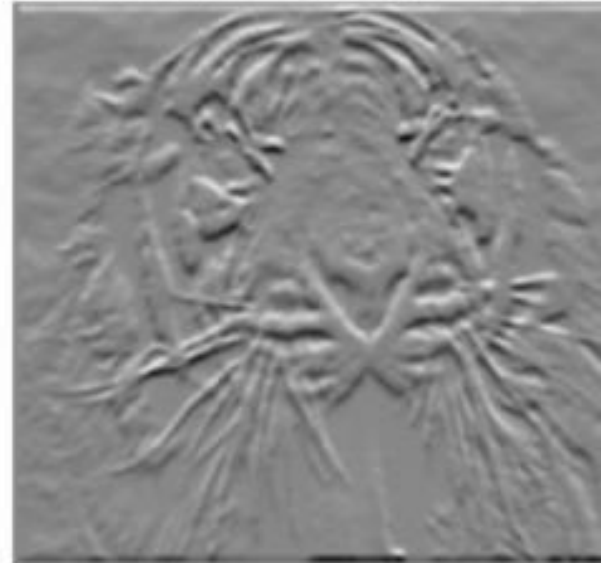
Módulo
del gradiente



Gradiente
en X



Gradiente
en Y



Extracción de características

Estimación local del gradiente



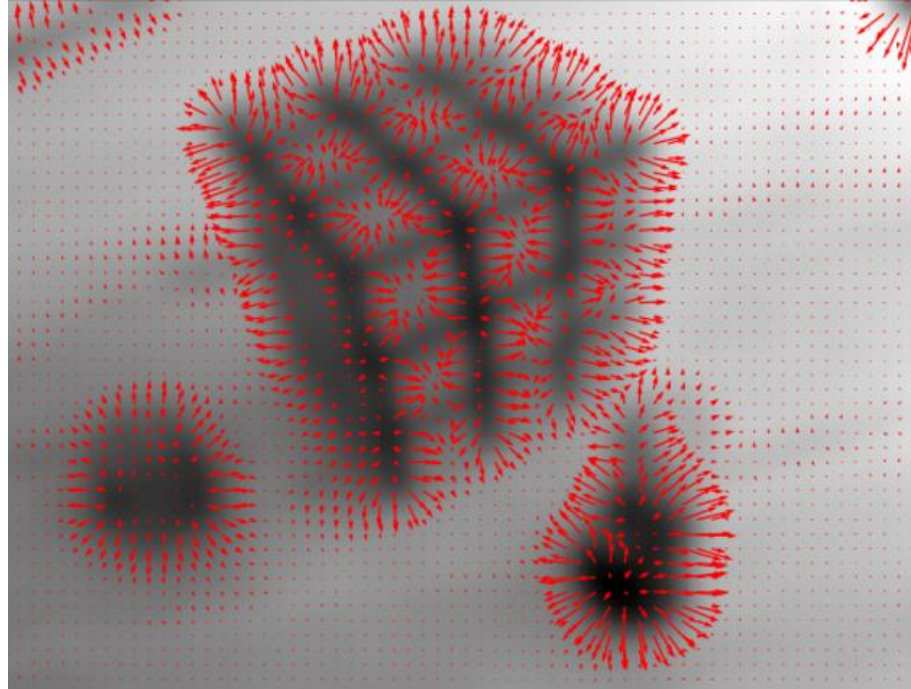
Máster en
Inteligencia
Artificial
UMU

Original

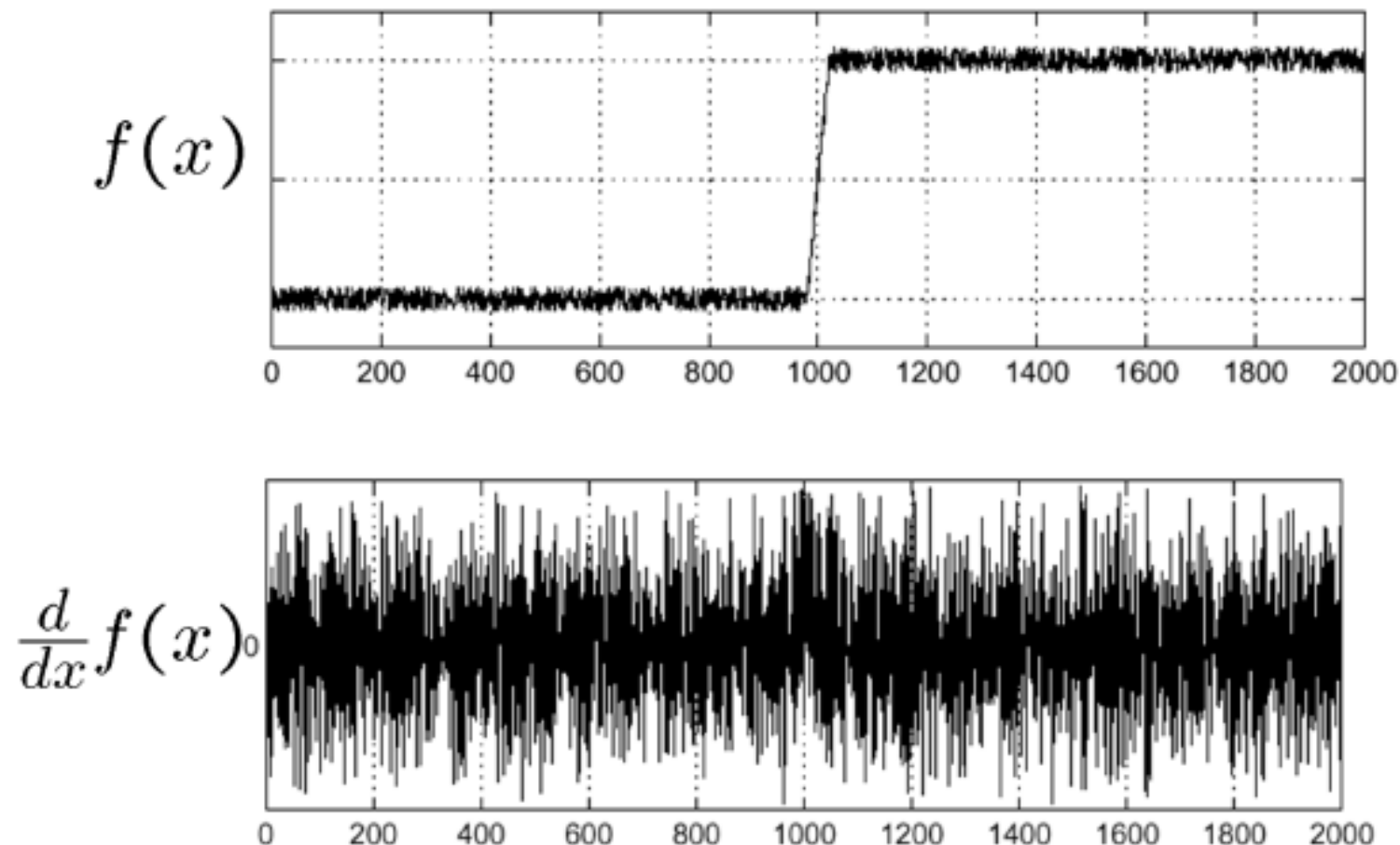


Suavizada
(Filtro gaussiano)

Gradientes
(vectores 2D)



- ¡Ojo! La simple diferencia finita ($G_x = I_x - I_{x-1}$) puede responder demasiado al ruido (ejemplo *scanline*):

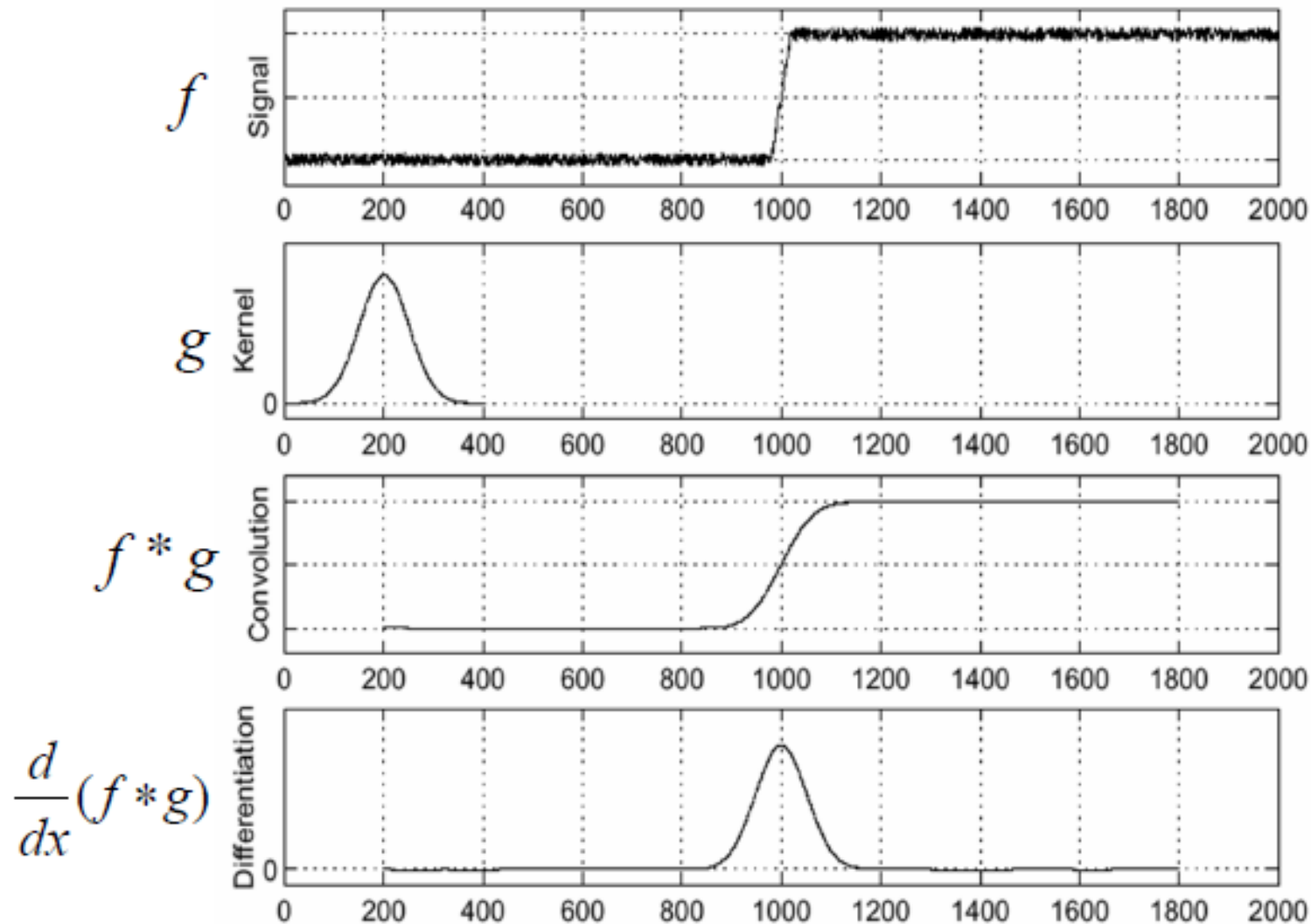


Extracción de características

Efectos del ruido en la estimación



- Solución: suavizar primero (p.e. con filtro Gaussiano)

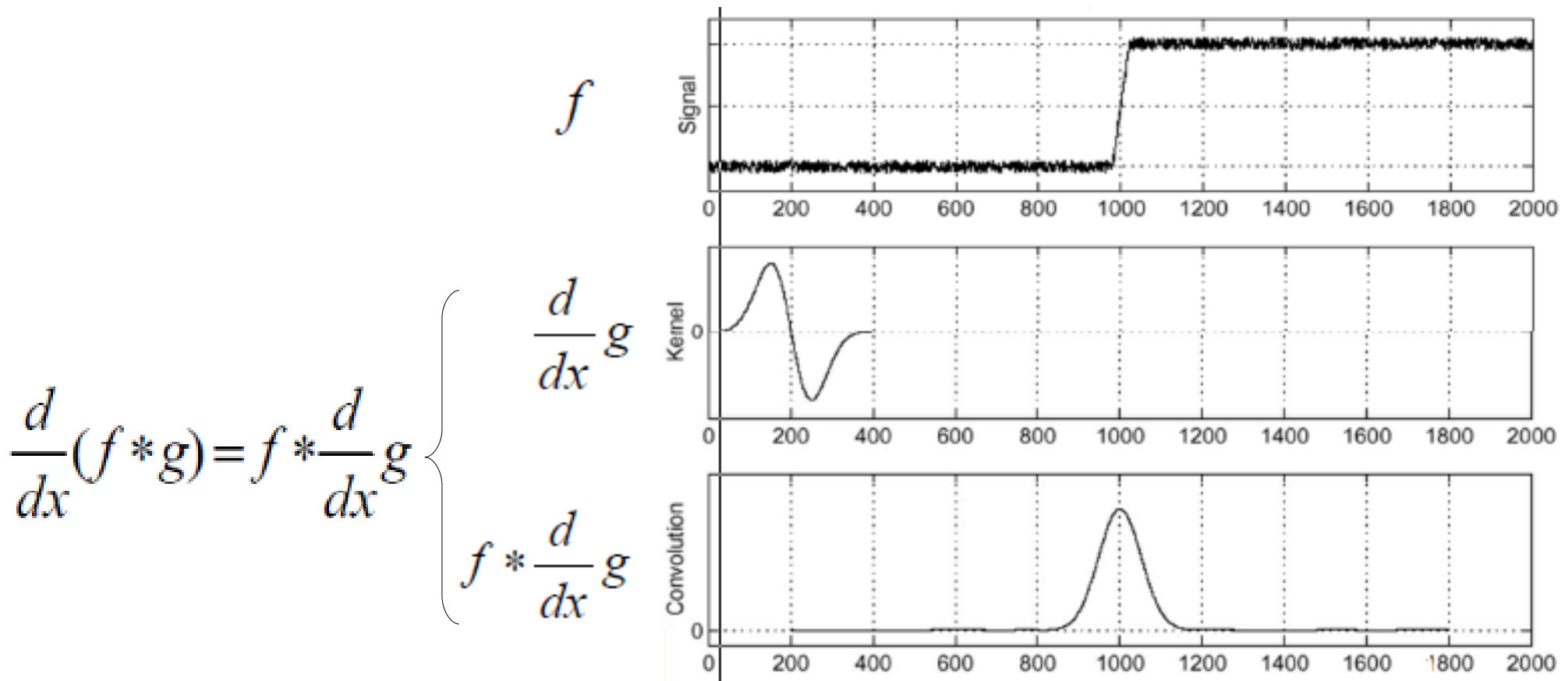


Extracción de características

Gradiente suavizado

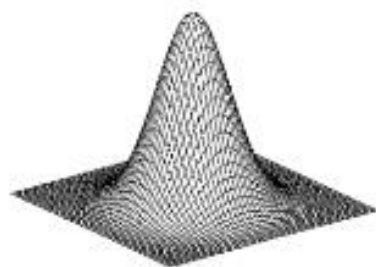


- Puesto que tanto la diferenciación como el suavizado son convoluciones, son asociativas, puede ser más económico combinarlas en una sola operación:

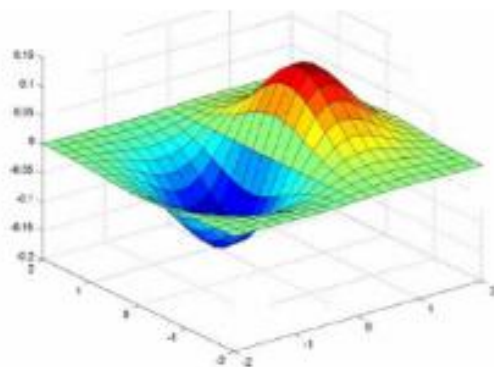
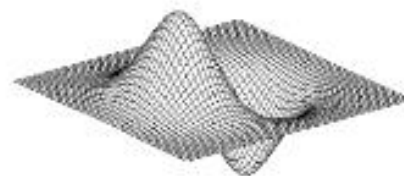


Extracción de características

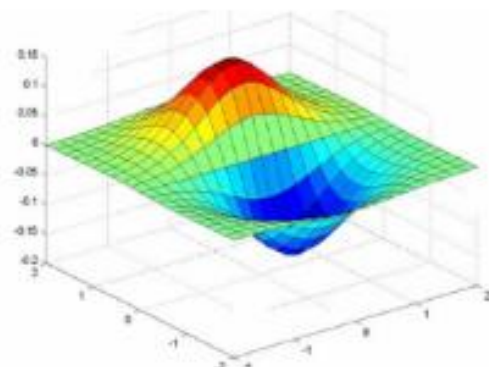
Gradiente suavizado (operador Sobel)



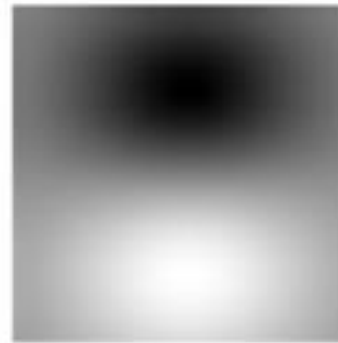
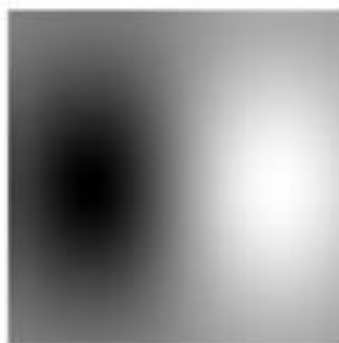
$$* [1 \ -1] =$$



x-direction



y-direction

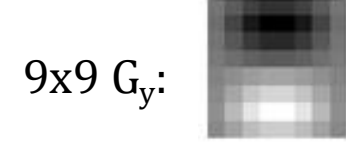
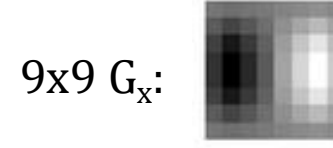
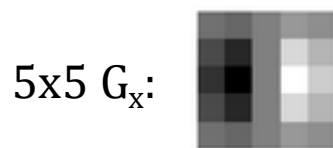


- El operador suavizado + gradiente puede hacerse a varias escalas (extensión del operador de Sobel):

$$\mathbf{J}_\sigma(\mathbf{x}) = \nabla[G_\sigma(\mathbf{x}) * I(\mathbf{x})] = [\nabla G_\sigma](\mathbf{x}) * I(\mathbf{x})$$

- Sólo hay que aplicar esta fórmula para generar las máscaras, decidiendo para cada escala σ el tamaño apropiado:

$$\nabla G_\sigma(\mathbf{x}) = \left(\frac{\partial G_\sigma}{\partial x}, \frac{\partial G_\sigma}{\partial y} \right)(\mathbf{x}) = [-x \quad -y] \frac{1}{\sigma^3} \exp \left(-\frac{x^2 + y^2}{2\sigma^2} \right)$$

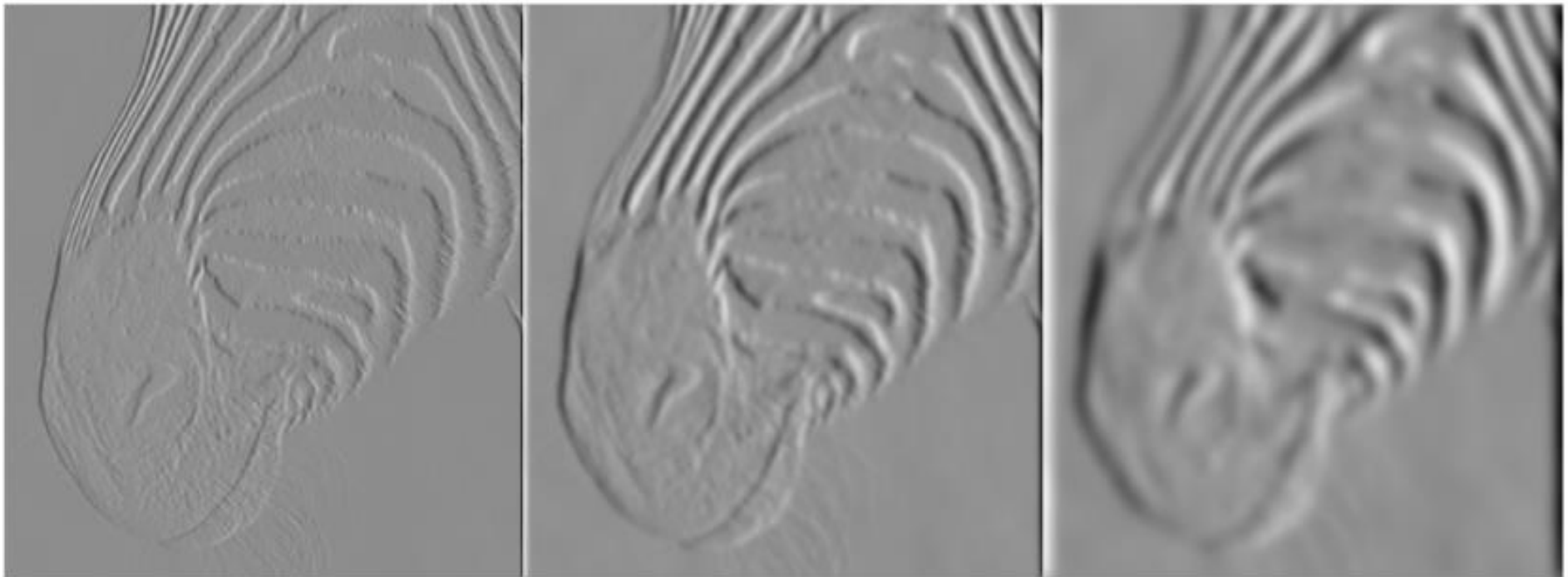


► Compromiso (*trade-off*) entre localización y suavizado:

Mejor localización



Menos ruido



Radio suavizado
1 píxel

Radio suavizado
3 píxeles

Radio suavizado
7 píxeles

Extracción de características

Resultados de diferentes estimadores



Máster en
Inteligencia
Artificial
UMU

► Comparación de resultados (módulo del gradiente):



Original



Roberts
(*cross*)

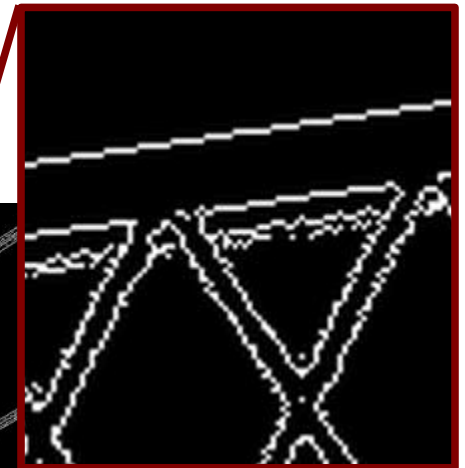
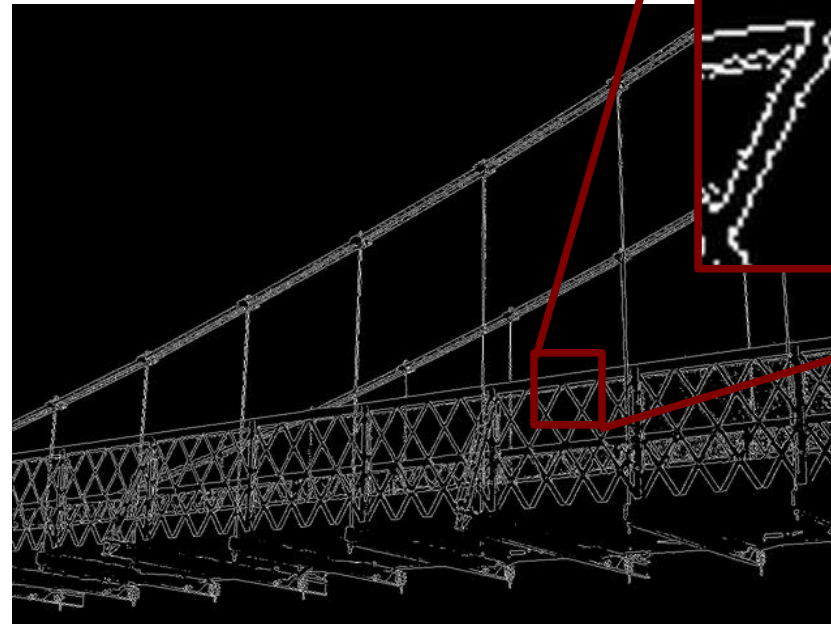


Prewitt



Sobel

- ▶ Ejemplo clásico de detección de bordes
 - Buscamos binarizar la imagen
 - 255 donde hay borde, 0 donde no:





- ▶ Problemas del simple umbralizado del módulo del gradiente:
 - Discontinuidad, acumulaciones inhomogéneas de píxeles activos (duplicidad), etc.
- ▶ Operador de Canny:
 - Intenta resolver los problemas de duplicidad y discontinuidad.
- ▶ Propiedades:
 - Buena detección.
 - Buena localización.
 - Mínima respuesta.

► Problemas del simple umbralizado del módulo del gradiente:

- Después del suavizado, queda demasiado “grueso”:
 - ¿Dónde está el borde realmente?
 - ¿Cómo enlazar para pasar de “puntos de borde” a “contornos”?



► Paso 1: Cálculo del gradiente



Original

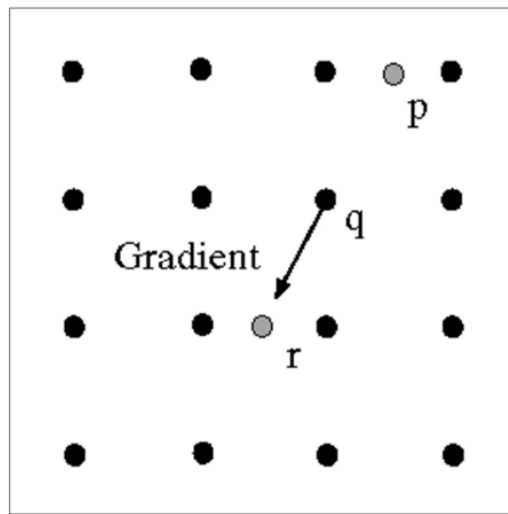
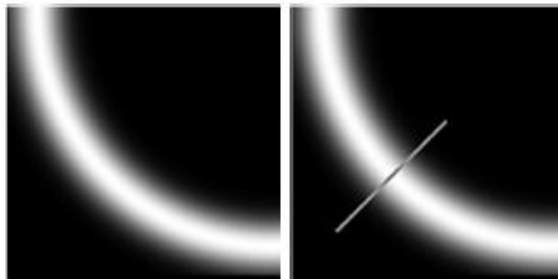


Módulo del
gradiente



Umbralizado
del módulo
del gradiente

- **Paso 2**: Supresión de no máximos
 - Utilizamos la dirección del gradiente

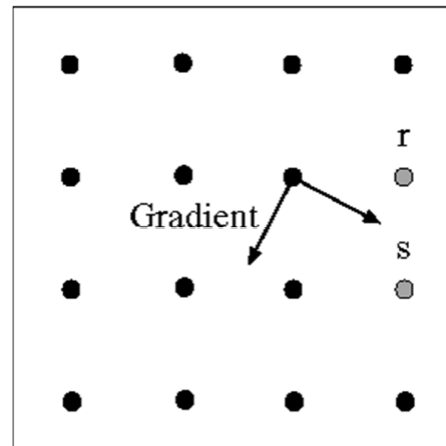


Hay un máximo en q si el valor es mayor que p y r (valores interpolados).

► Paso 3: Histéresis



Original



Los bordes son perpendiculares al gradiente: si el píxel marcado es de borde, el siguiente será *r* o *s* (*edge linking*)



Umbral alto
(bordes “fuertes”)



Umbral bajo
(bordes “débiles”)



Histéresis

► Resumen de pasos del operador:

- **Cálculo de gradiente** sobre imagen suavizada (p.e., Sobel): G_x , G_y , magnitud (raíz de suma de cuadrados) y orientación (arcotangente).
- **Supresión de no-máximos**: Se buscan y marcan sólo los máximos locales del valor de la magnitud del gradiente en la orientación marcada por el mismo. Así se obtienen “bordes delgados”.
- **Histéresis**: en el proceso anterior, para que un máximo local se marque como respuesta, tiene que cumplirse:
 - Que la magnitud sea mayor que un umbral **High**.
 - O bien que esté entre un umbral **Low** y el umbral **High**, siempre que el píxel esté conectado a un píxel ya activo.
 - Pero si la magnitud es menor que **Low**, el píxel se descarta directamente.

https://docs.nvidia.com/vpi/algo_canny_edge_detector.html

Extracción de características

Operador de Canny



Máster en
Inteligencia
Artificial
UMU

- La **elección de la escala** depende de los tipos de bordes que nos interesen:



Original

$\sigma=1.0$

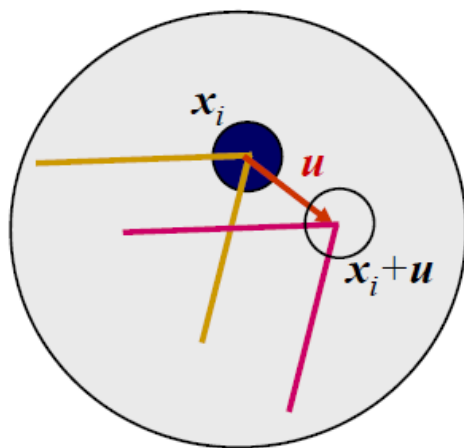
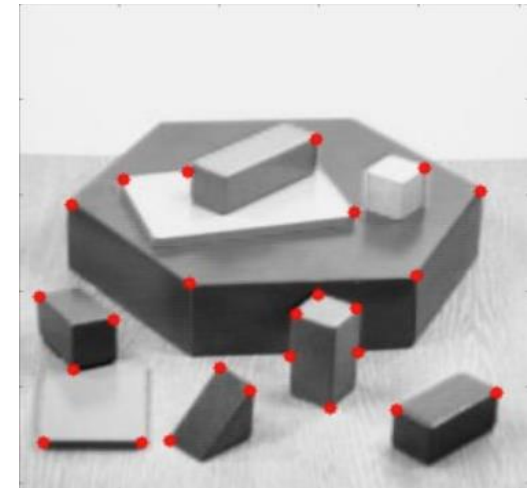
$\sigma=2.0$

Extracción de características

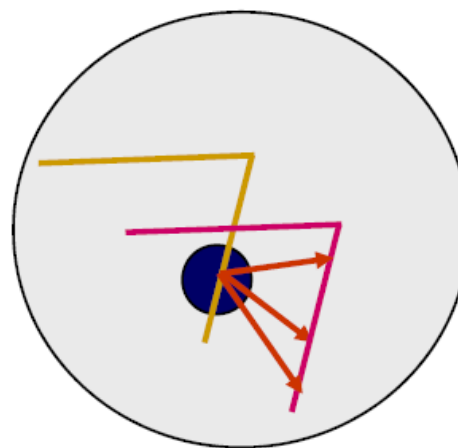
Métodos de extracción de puntos



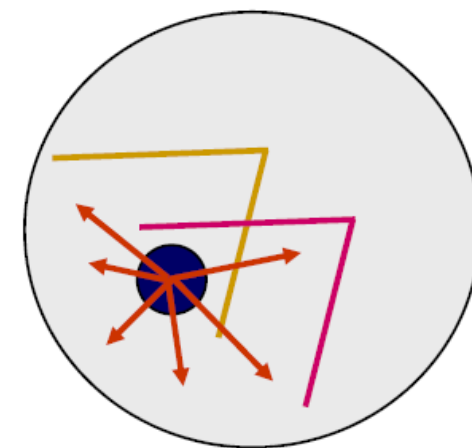
- ¿Cómo encontrar localizaciones de la imagen para las cuales podremos encontrar correspondencias con fiabilidad en otra imagen?



(a) Buena localización
2D (*corner-like*)



(b) Mala localización
1D (problema de apertura)

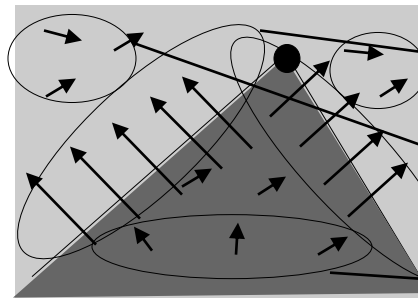


(c) Mala localización
2D (sin textura)

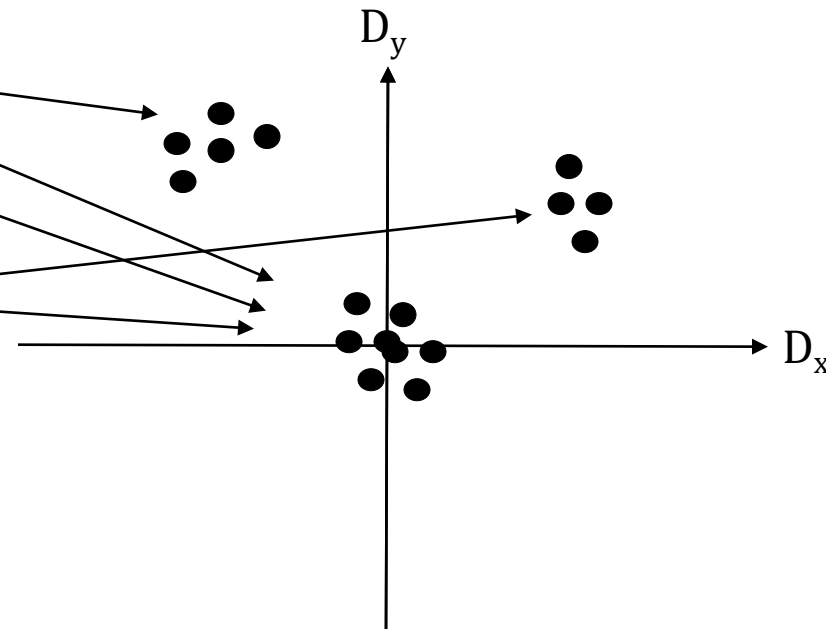
Imagen 1 ●
Imagen 2 ○

► Estudio de valores del gradiente en un entorno local:

Entorno local
de una esquina:

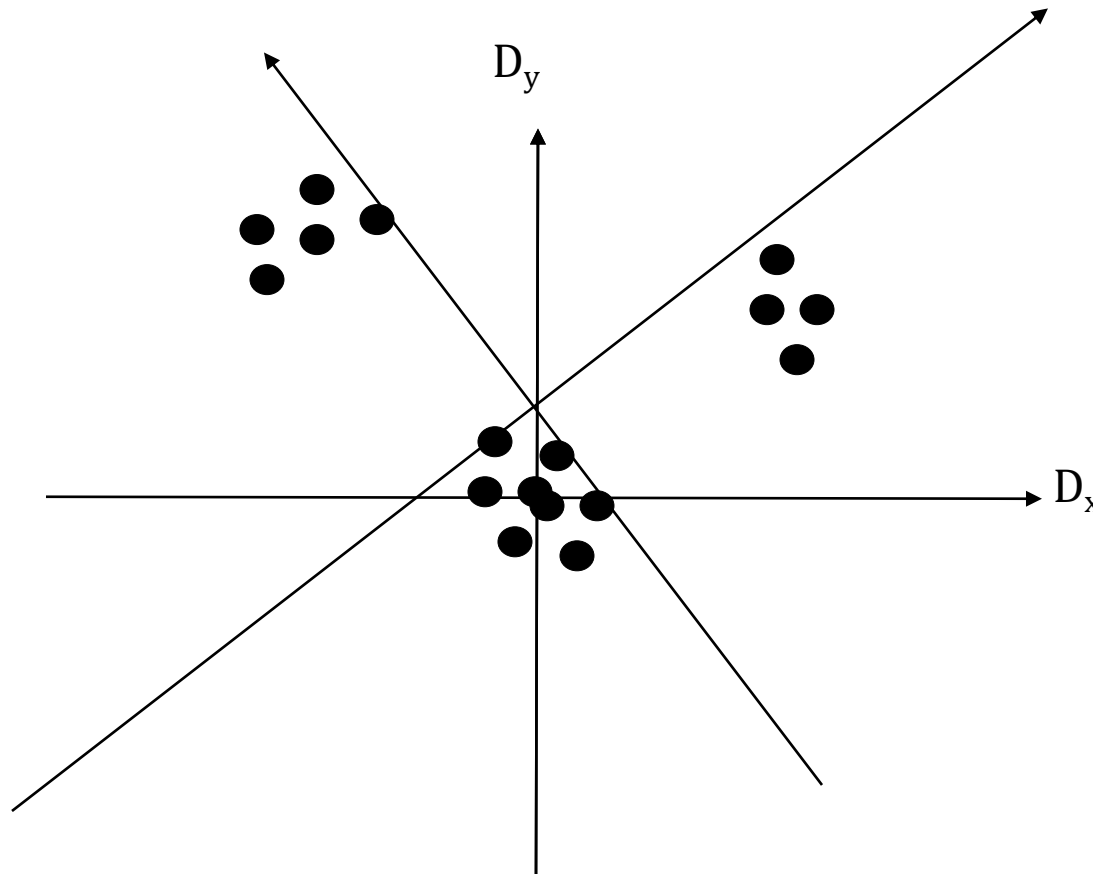


Dominio de la imagen
(valores de gris)



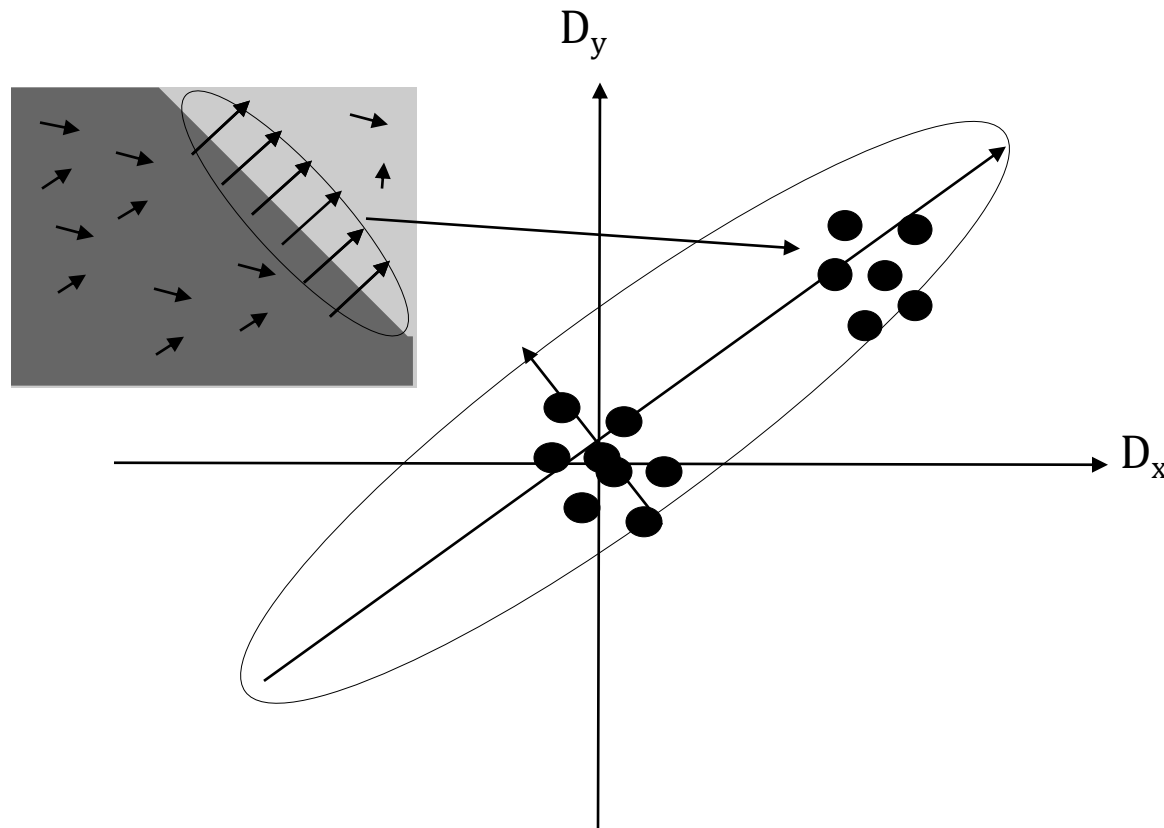
Dominio del gradiente
(nube de puntos)

- Caracterización de las direcciones principales de la distribución de los vectores gradiente:



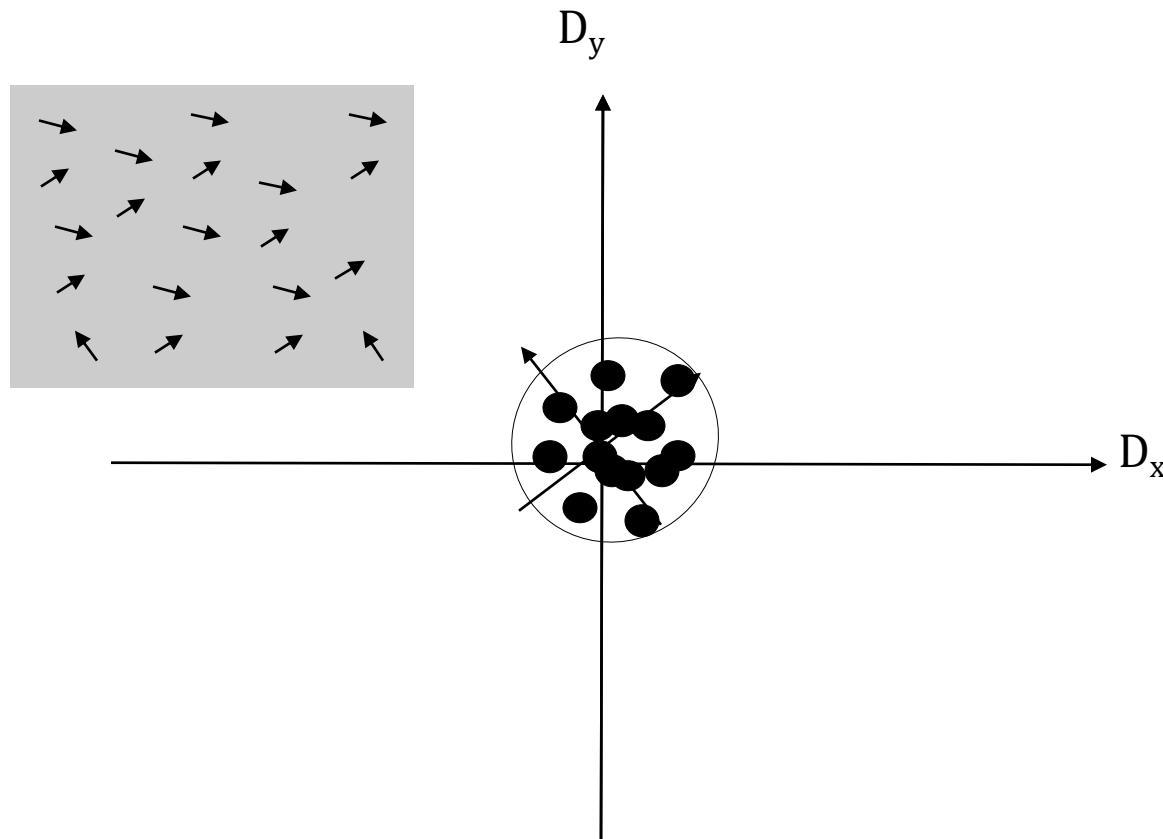
Dos modos de
variación apreciables

► Comparar con un borde recto...:



Un sólo modo de
variación apreciable

► ... y con una zona más o menos “lisa”:



No modo de
variación apreciable

Extracción de características

Método de extracción de “esquinas”

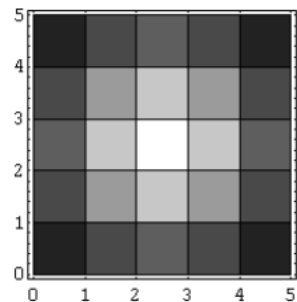


► Resumen de la idea:

- En el entorno local de cada pixel (ventana), calcular la siguiente matriz, que caracteriza la distribución espacial del gradiente:

$$A = \sum_u \sum_v w(u, v) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} \langle I_x^2 \rangle & \langle I_x I_y \rangle \\ \langle I_x I_y \rangle & \langle I_y^2 \rangle \end{bmatrix},$$

(Opcional)
Ventana de
“weighting”



I_x = derivada parcial en X

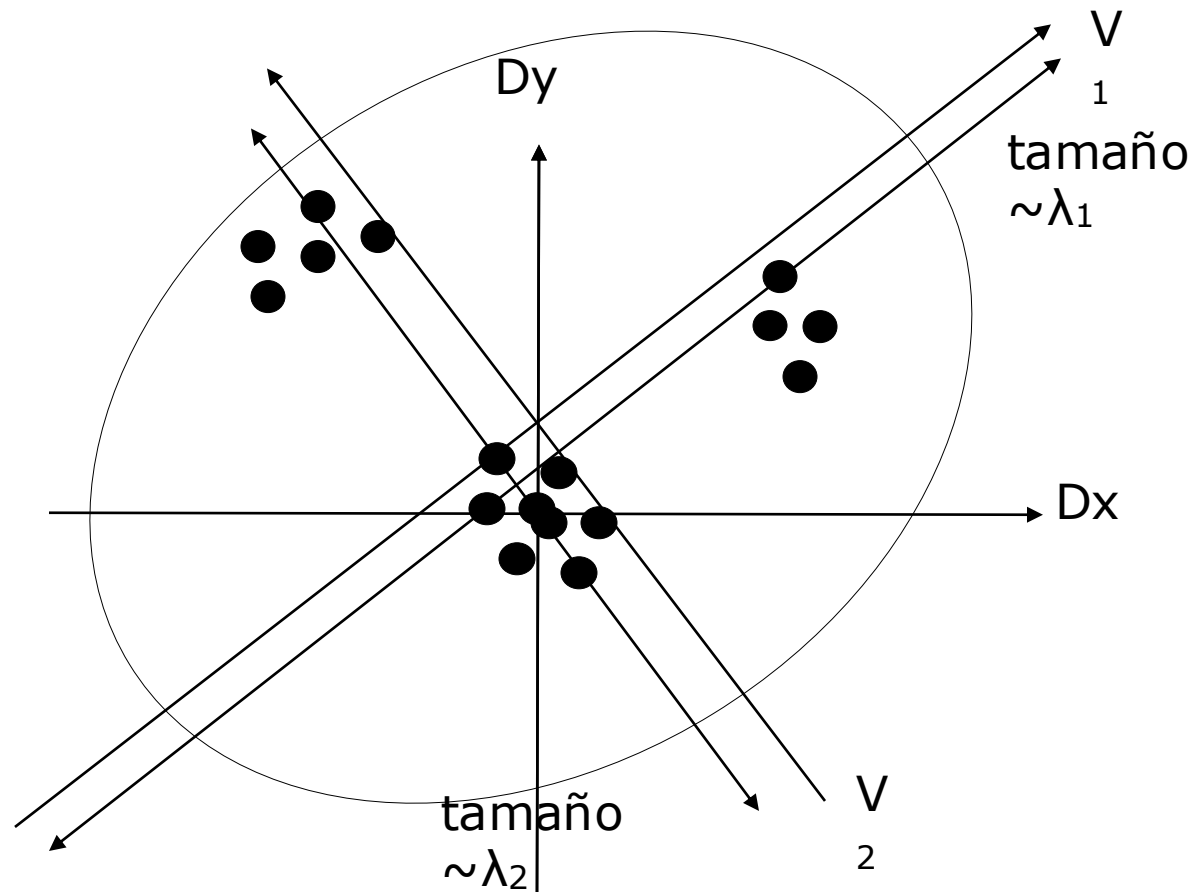
I_y = derivada parcial en Y

Extracción de características

Método de extracción de “esquinas”



- Análisis de valores λ_1 y λ_2 y vectores propios v_1 y v_2 de la matriz A :



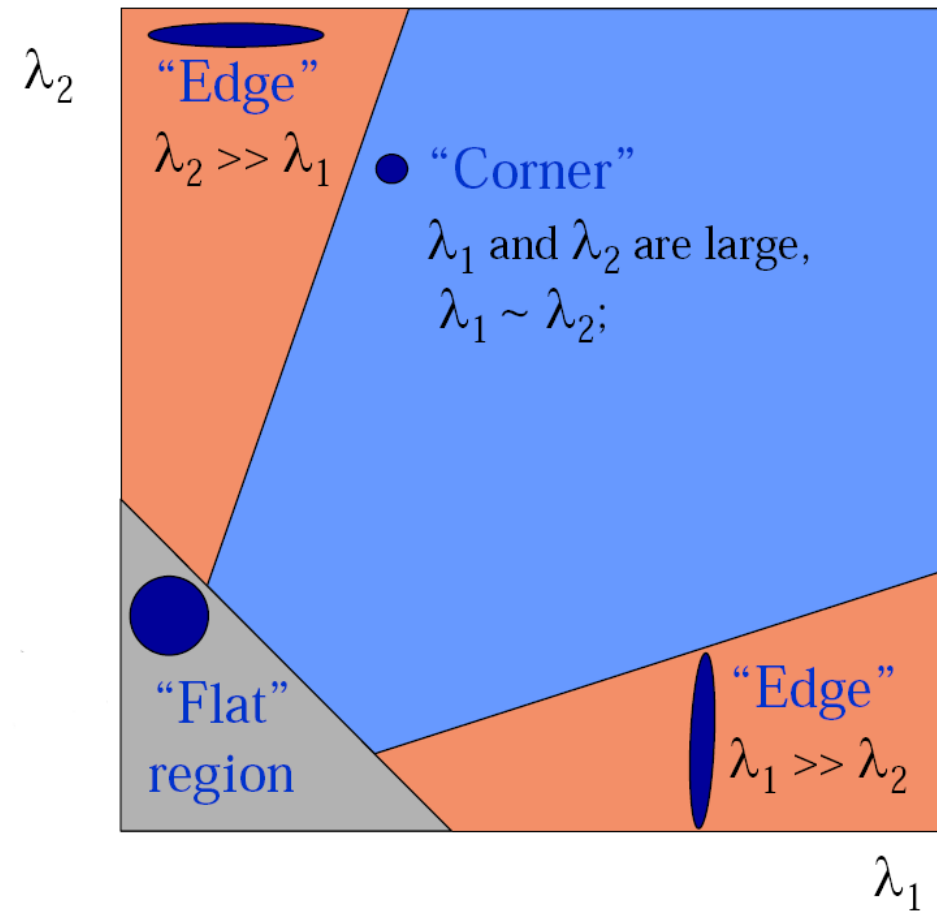
Extracción de características

Método de extracción de “esquinas”



► Para cada punto de interés (esquina, o *corner*), A debería tener dos valores propios “grandes”. Así:

- Si $\lambda_1 \sim 0$ y $\lambda_2 \sim 0 \rightarrow$ No hay característica de interés en este entorno (zona “lisa”).
- Si $\lambda_1 \gg 0$ y $\lambda_2 \sim 0 \rightarrow$ Simplemente hay un borde apreciable en este entorno (pero no una esquina).
- Si $\lambda_1 \gg 0$ y $\lambda_2 \gg 0 \rightarrow$ Se ha detectado una esquina en este punto.



Extracción de características

Ejemplo de uso del operador de Harris



Máster en
Inteligencia
Artificial
UMU

► Imágenes de entrada originales:



Extracción de características

Ejemplo de uso del operador de Harris



- Salida operador respuesta: $\lambda_{min} \approx \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2} = \frac{Det(A)}{Tr(A)} \quad (= \frac{Det(D)}{Tr(D)}, \text{ con } A = V^T D V)$

- Naranja→más alto:

Eigen-descomposición de la matriz y A , λ_1 y λ_2 son los elementos de la diagonal de D

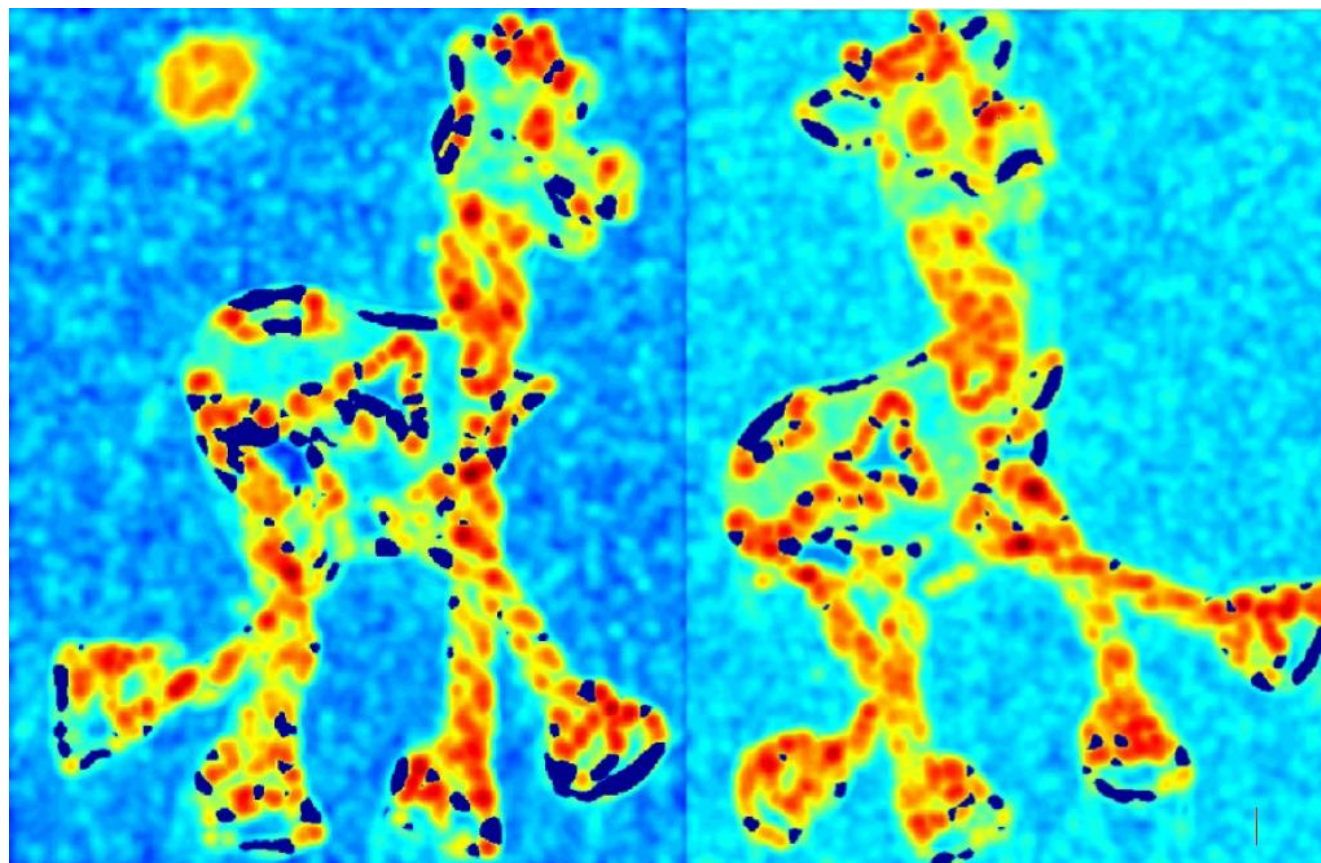
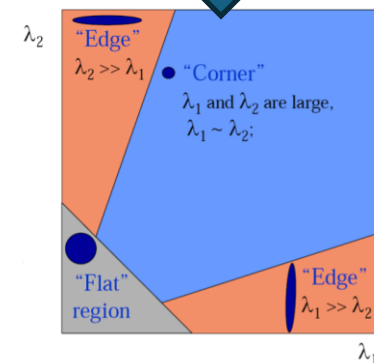
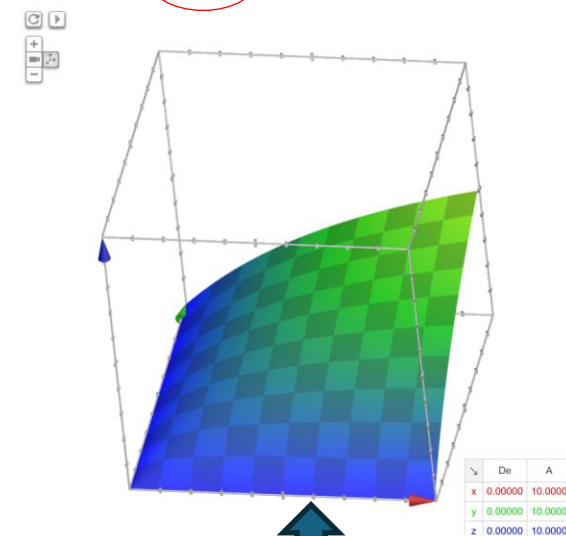


Gráfico de $x*y/(x+y)$

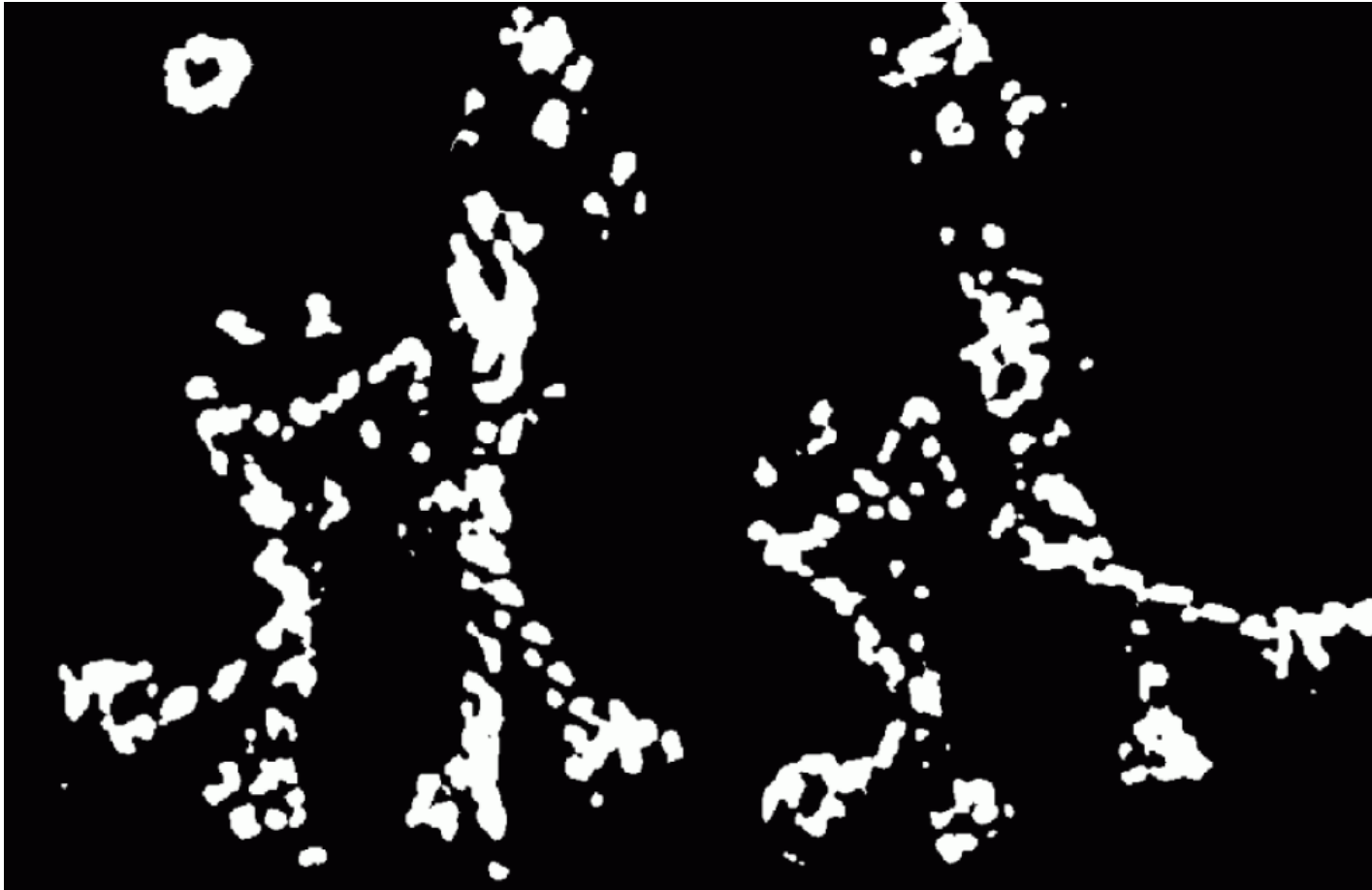


Extracción de características

Ejemplo de uso del operador de Harris



► Umbralizado del valor de respuesta:

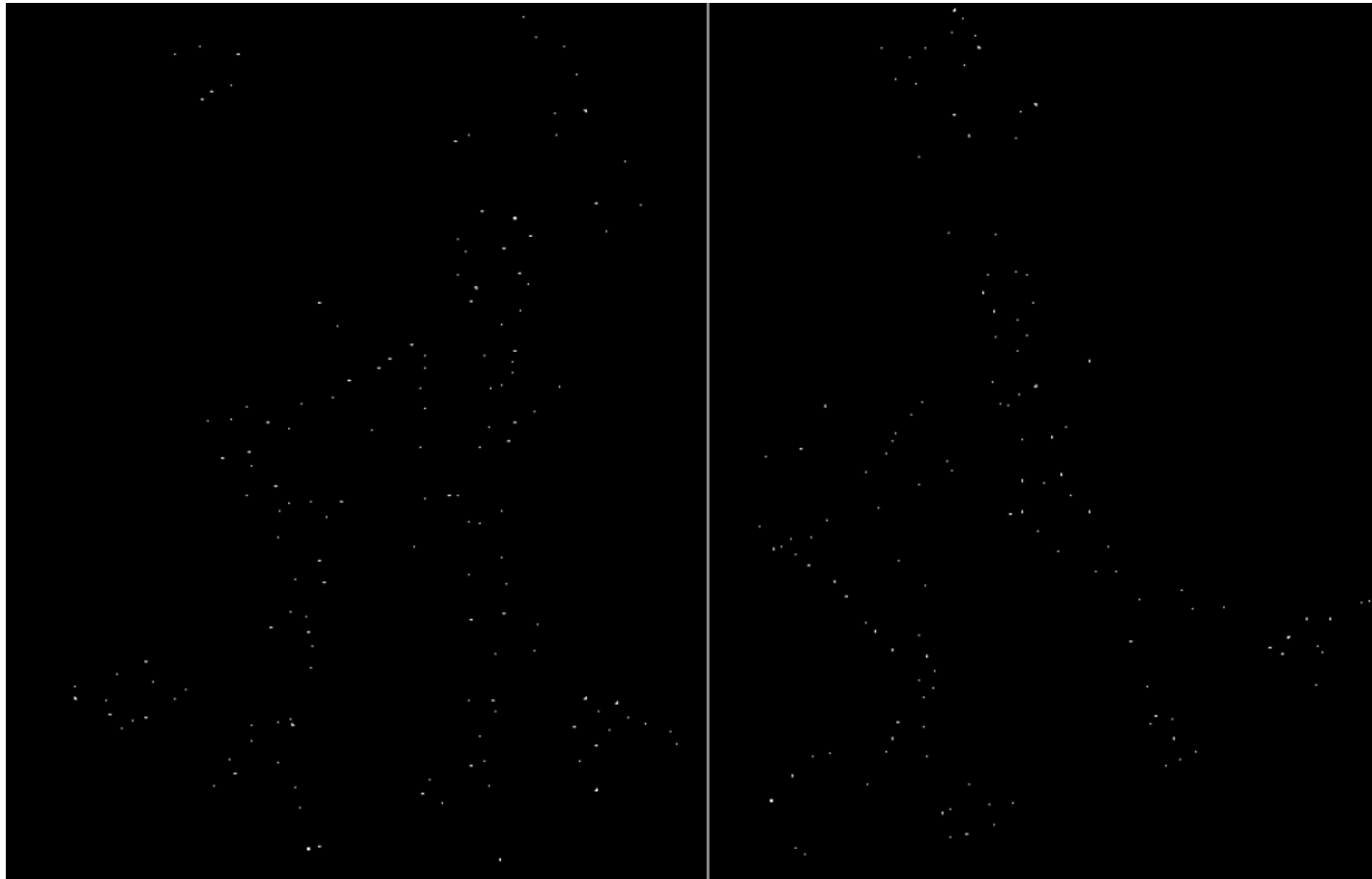


Extracción de características

Ejemplo de uso del operador de Harris



► Detección de máximos locales:



Extracción de características

Ejemplo de uso del operador de Harris



Máster en
Inteligencia
Artificial
UMU

► Esquinas detectadas sobre imágenes originales:



https://docs.nvidia.com/vpi/algo_harris_corners.html

Extracción de características

Supresión de no máximos adaptativa

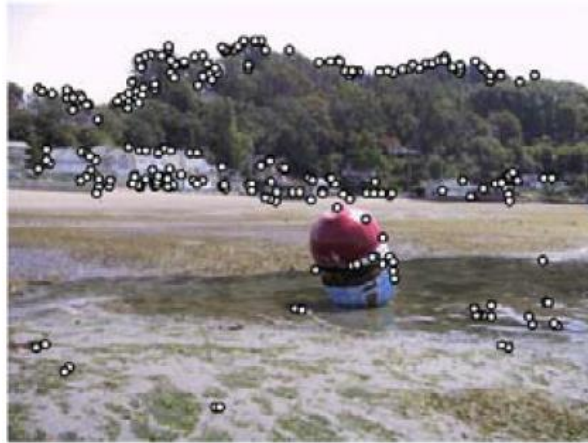


Máster en
Inteligencia
Artificial
UMU

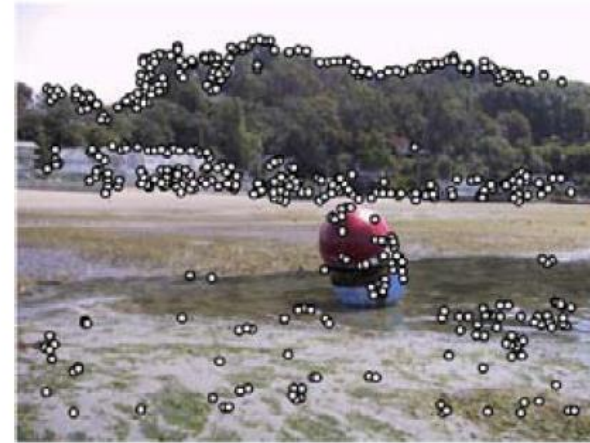
► ANMS (*Adaptive Non-maxima Suppression*)

- Se ordenan puntos (que superan un umbral) por “radio de supresión” (distancia al píxel de respuesta mayor más cercano):

250 mayores
respuestas



500 mayores
respuestas



250 mayores
respuestas,
Radio supresión
24 píxeles



500 mayores
respuestas,
Radio supresión
16 píxeles



Extracción de características

Hessiano local



- Usando una idea similar a la del detector de Harris anterior, pero sustituyendo la matriz A con la matriz H (ambas de tamaño 2×2) de segundas derivadas de la imagen, en lugar de la calculada con gradientes:

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

- Este es exactamente el Hessiano de la aproximación local cuadrática al parche de la imagen (aquí consideramos $(x,y)=(0,0)$ el centro del parche):

$$im(x, y) = f(x, y) \approx f(0, 0) + \nabla f(0, 0) \cdot (x, y) + \frac{1}{2}(x, y) \cdot H \cdot (x, y)^T$$

- Después, el análisis de valores propios es similar: si hay una fuerte asimetría en la curvatura local, se rechaza el punto (por no tener estructura suficiente)

Enlaces interesantes sobre
aproximaciones de Taylor:

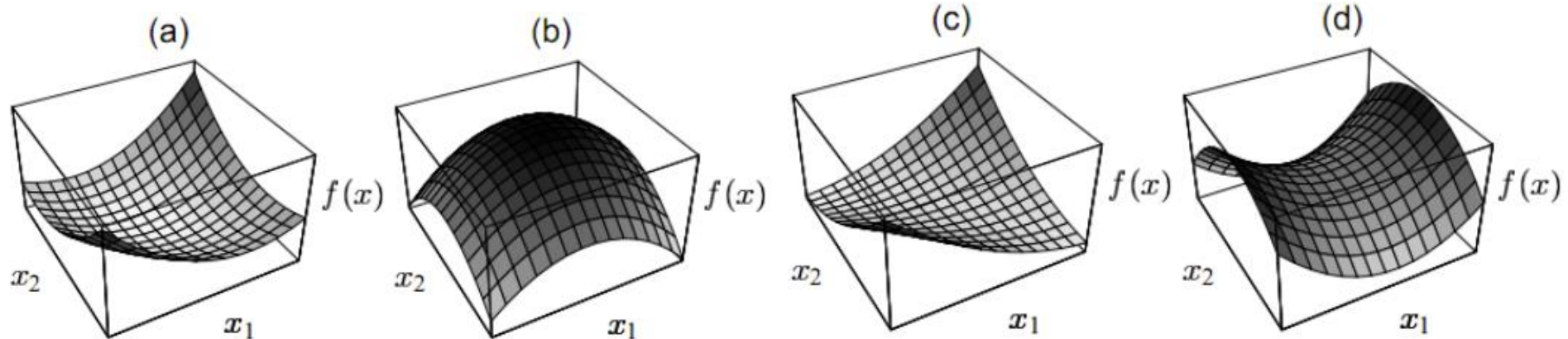
[\(1D\)](#) [\(2D\)](#)

Extracción de características

Hessiano local



- ▶ (a) y (b): ambos valores propios λ_1 y λ_2 positivos / negativos, y lejanos de cero $\rightarrow$ *blob* oscuro / claro, interesa.
- ▶ (c): un valor propio positivo, otro cercano a cero $\rightarrow$ problema de apertura, no interesa.
- ▶ (d): ambos valores propios bastante mayores que cero en valor absoluto, pero uno positivo y otro negativo $\rightarrow$ punto de silla (*saddle point*), interesa.
- ▶ Para otro tipo de zonas (planas, con o sin pendiente), ambos valores propios cercanos a cero $\rightarrow$ no interesa.

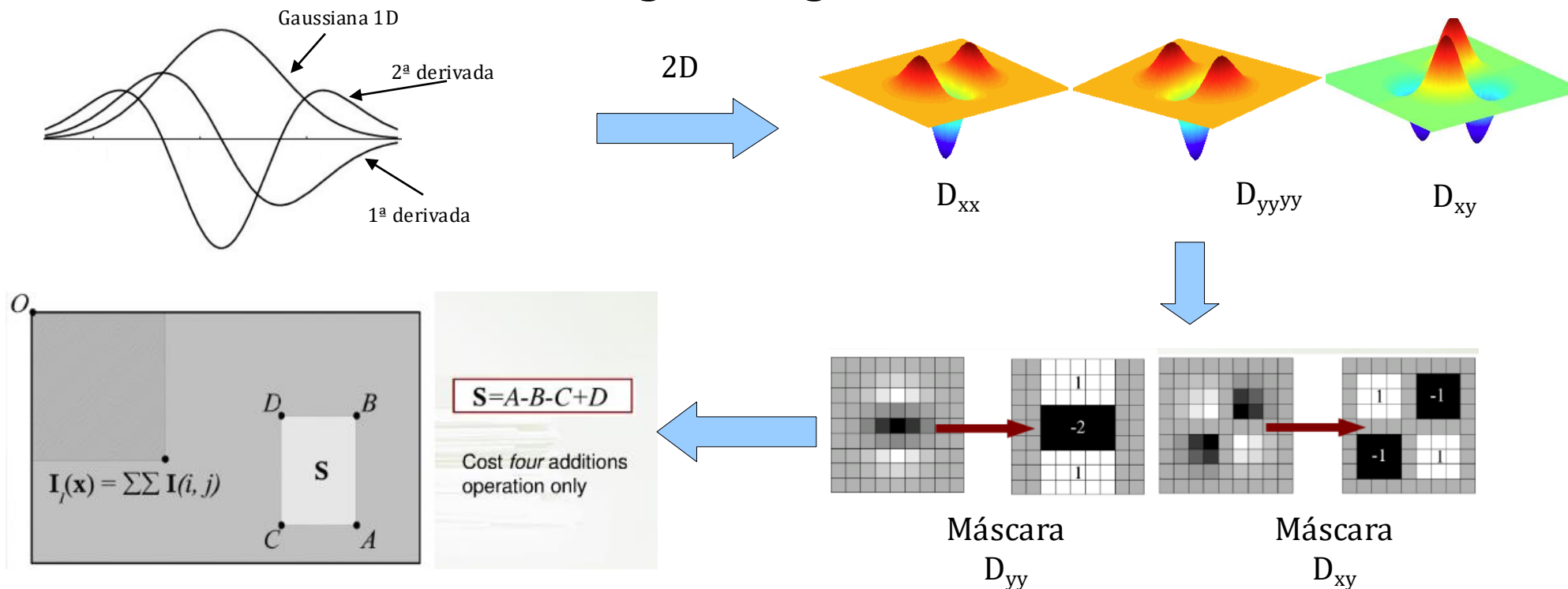


Extracción de características

Hessiano local



- Suele usarse más para entornos en escala variable.
- P.e. el detector **SURF**, (*Speeded Up Robust Features*) que trabaja con máscaras de sumas y restas en áreas rectangulares/cuadradas para estimar las segundas derivadas (en lugar de con filtros gaussianos), por ser fácilmente aplicables de modo multiescala usando la técnica de la imagen integral:

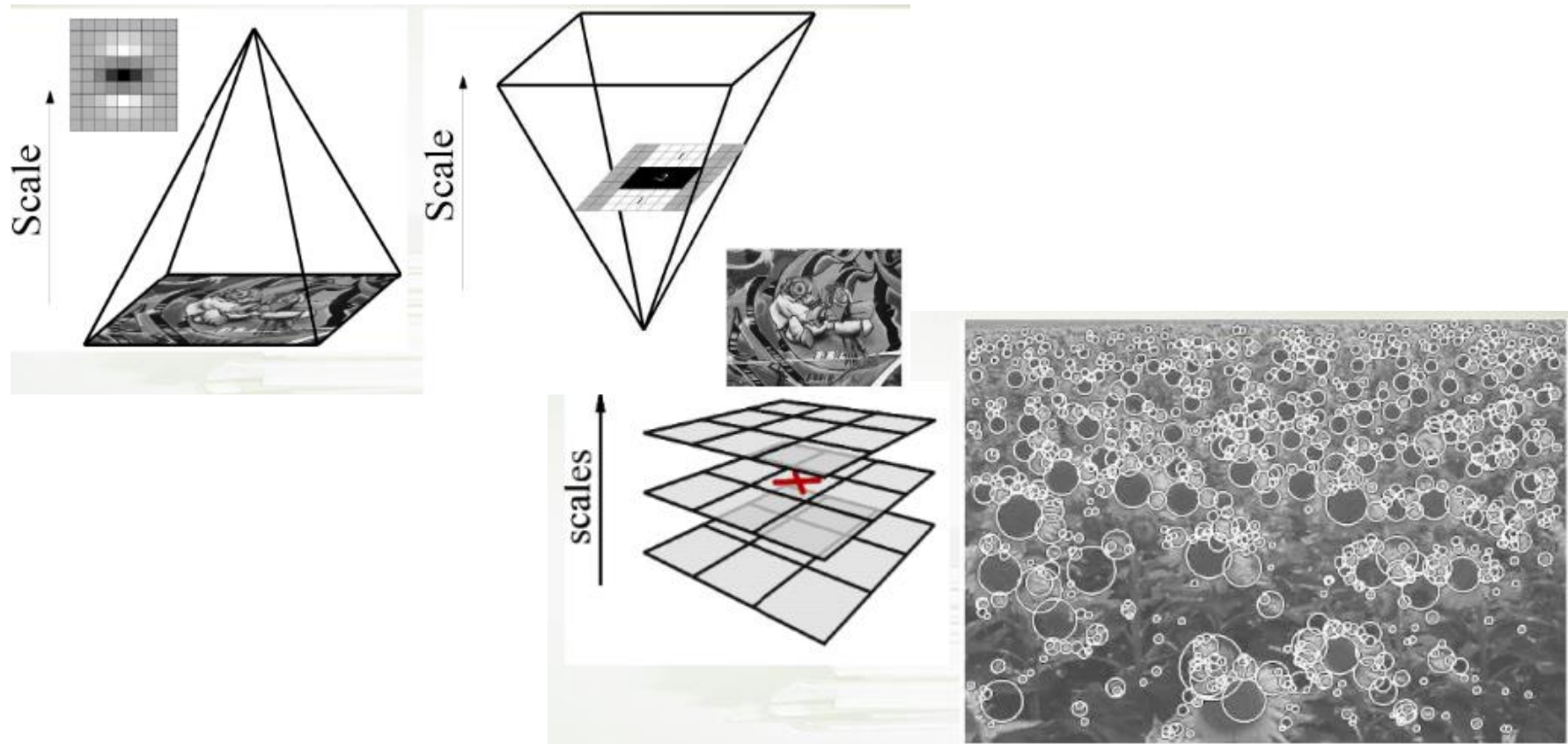


Extracción de características

Hessiano local



► Ejemplo detección multiescala SURF:



► En realidad, bastante análogo a SIFT, que veremos en más detalle más adelante.

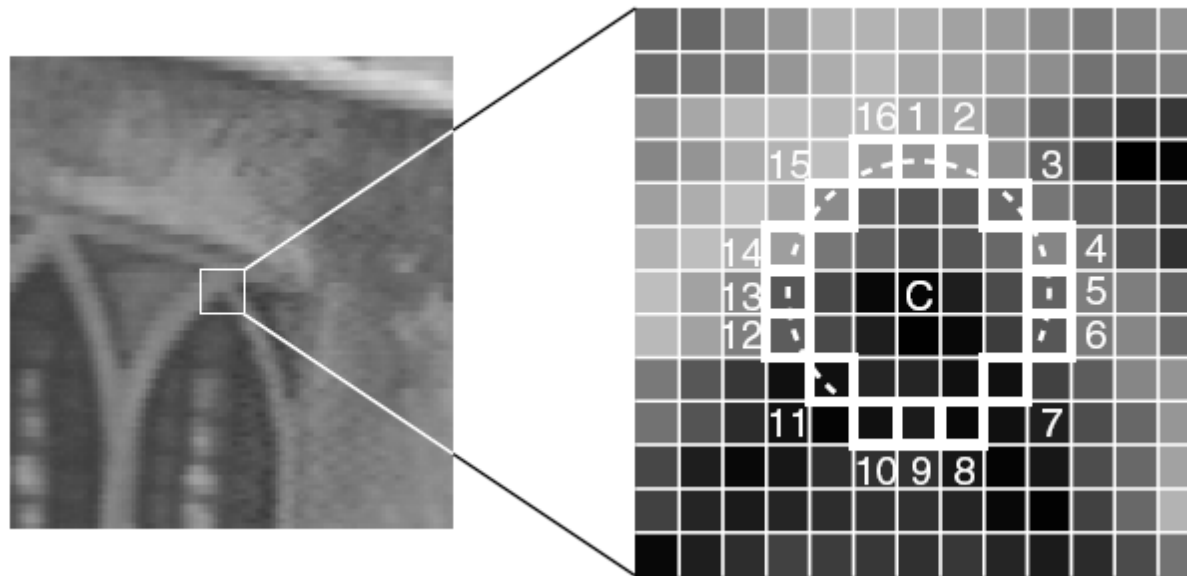
Extracción de características

Más detectores de esquinas: el operador FAST



- ▶ **FAST:** *Features from Accelerated Segment Test.*
- ▶ Se examina el círculo de Bresenham centrado en torno a un píxel (radio 3, 16 píxeles).
- ▶ Si la intensidad de al menos 12 píxeles contiguos de esos 16 está siempre un determinado umbral por encima (o por debajo) del valor de la intensidad en el centro de la máscara, se considera que hay una esquina.
- ▶ El orden de escaneo no es circular, sino, por eficiencia: N, S, E y W primero, y si al menos tres de estos píxeles no cumplen la condición, el píxel se descarta sin más procesamiento.

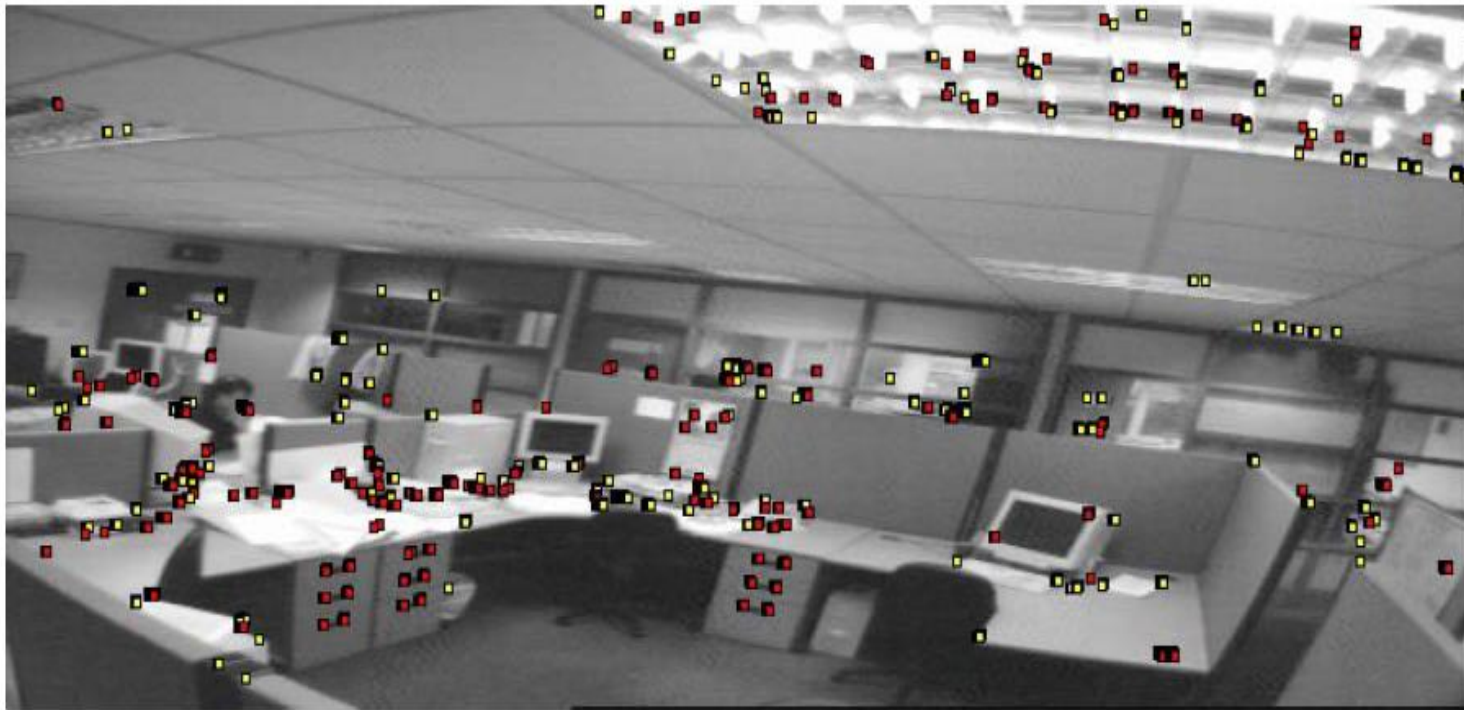
► Ilustración del método:



El método es **extremadamente rápido** (puede implementarse con tiempos de ejecución por debajo de 1 *ms* por frame para tamaños de imagen incluso muy grandes).

https://docs.nvidia.com/vpi/algo_fast_corners_detector.html

► Ejemplo de funcionamiento:



Puntos rojos: Esquinas “positivas”

Puntos amarillos: Esquinas “negativas”

[Ejemplo online](#) de tracking de puntos (tipo Harris, hessiano, FAST → *sparse optical flow*): (*Video*→*OpticalFlow example*)

Extracción de características

SIFT (*Scale Invariant Feature Transform*)



- ▶ Último y más potente método de extracción de características “*hand-engineered*” (David Lowe, 2004), previo a la “revolución *Deep Learning*” (~2012).
- ▶ Extracción de *features* **robusta, invariante a escala, ...**
- ▶ ... **más un descriptor asociado** invariante a escala, rotación, cambio de iluminación y ligeras deformaciones.
- ▶ En el *paper* original se describe también una aplicación al reconocimiento de objetos en condiciones relativamente incontroladas (escala y posición del objeto completamente desconocidas, *cluttered background*, etc.)
- ▶ Sigue siendo el más utilizado para detectar objetos particulares **sin entrenamiento**.

Extracción de características

Motivación



Máster en
Inteligencia
Artificial
UMU

► Ejemplos de localización y reconocimiento de objetos:



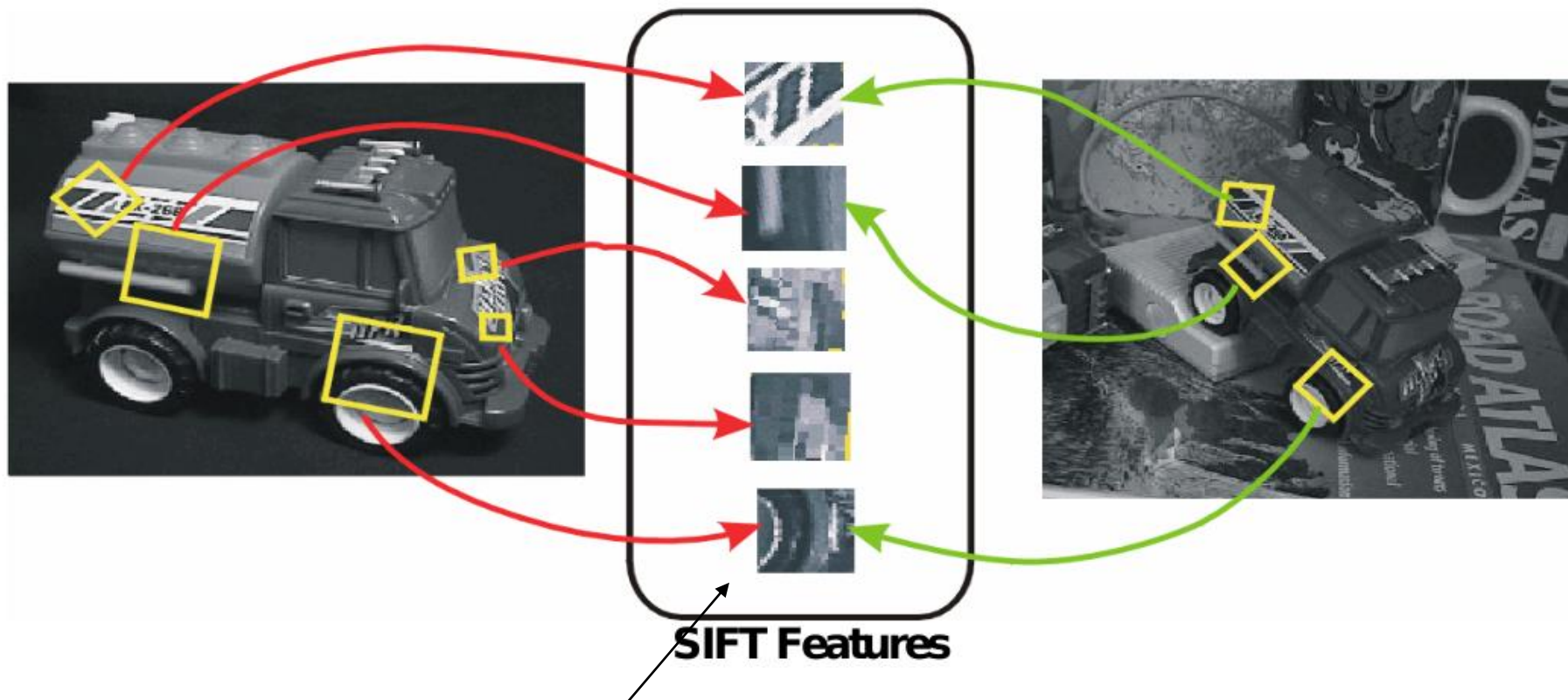
► Obsérvense las ocultaciones, los cambios de escala, el *background* desconocido...

Extracción de características

Idea para el reconocimiento de objetos



- Matching entre descriptores SIFT “imagen de entrada” ↔ “modelo”:



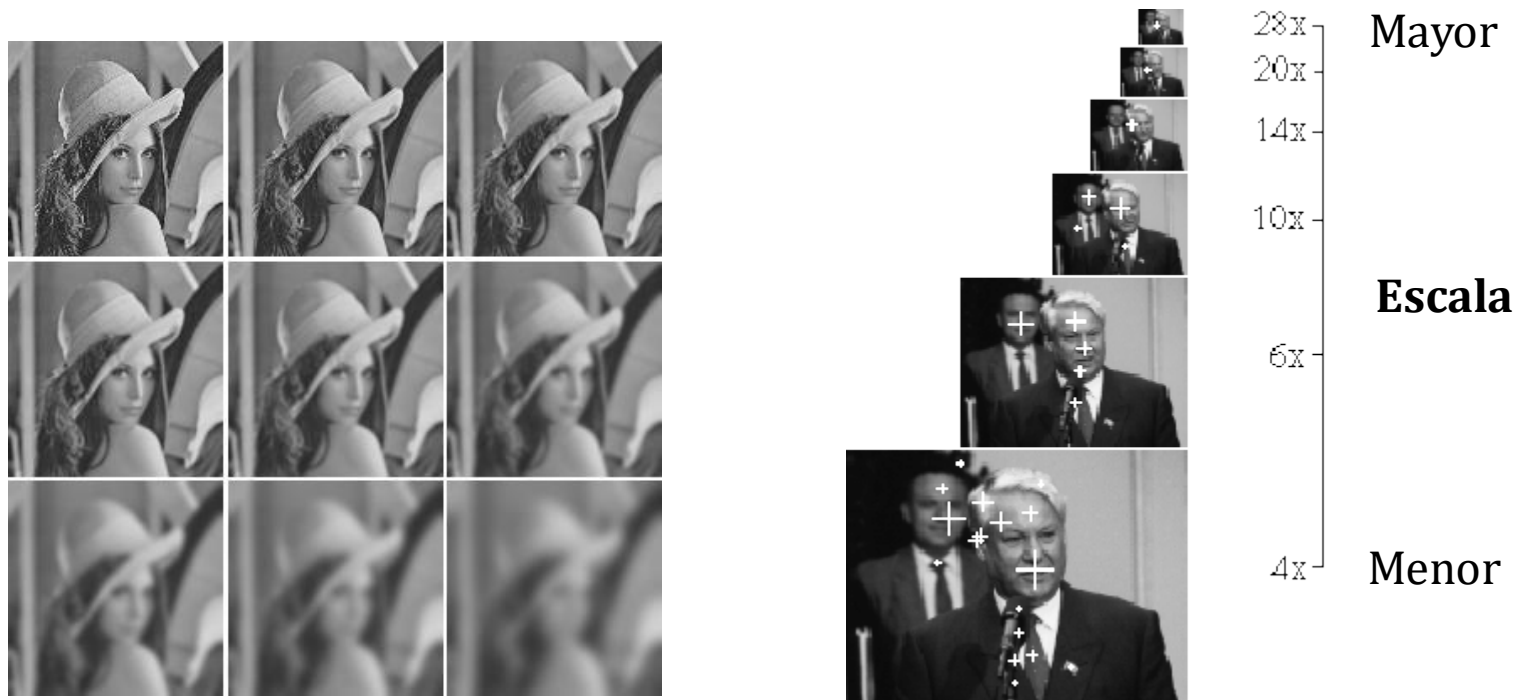
En realidad, los descriptores serán algo más complejos que el simple parche de imagen, pero aquí se recoge la idea de las invarianzas a escala/rotación

Extracción de características

Detección de extremos en el espacio de escala



- ▶ Intentar localizar lugares de la imagen de forma repetible frente a cambios de escala, orientación y luminosidad.
- ▶ Aproximación: buscar repetidamente a distintas escalas:

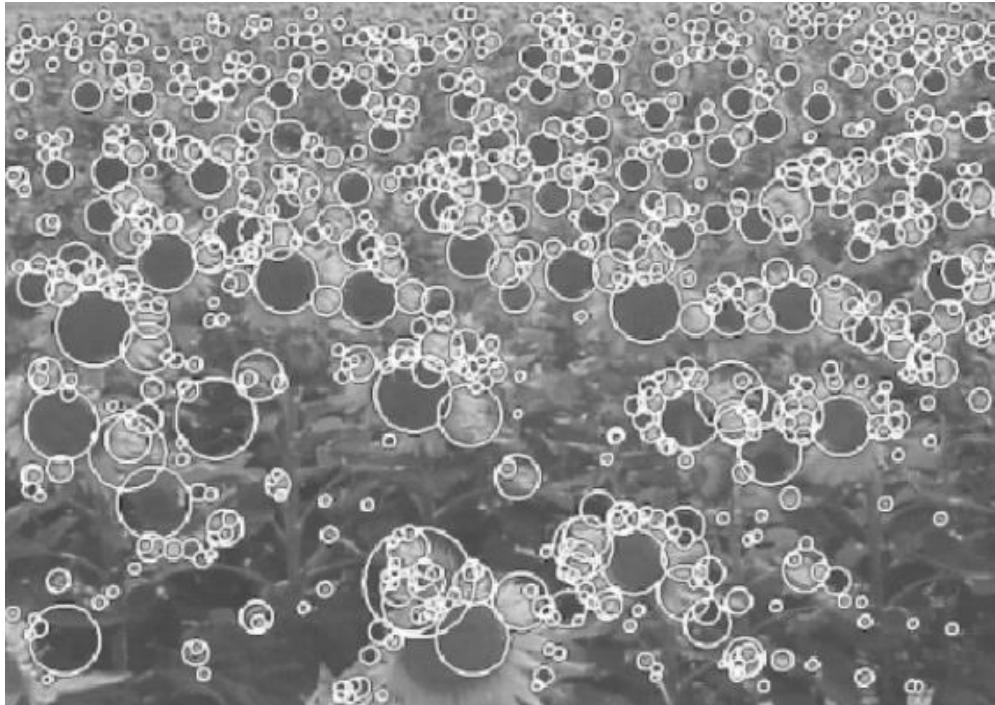


Extracción de características

Detección de extremos en el espacio de escala



- Nos interesa buscar *blobs* de imagen que sean repetibles a distintas escalas:



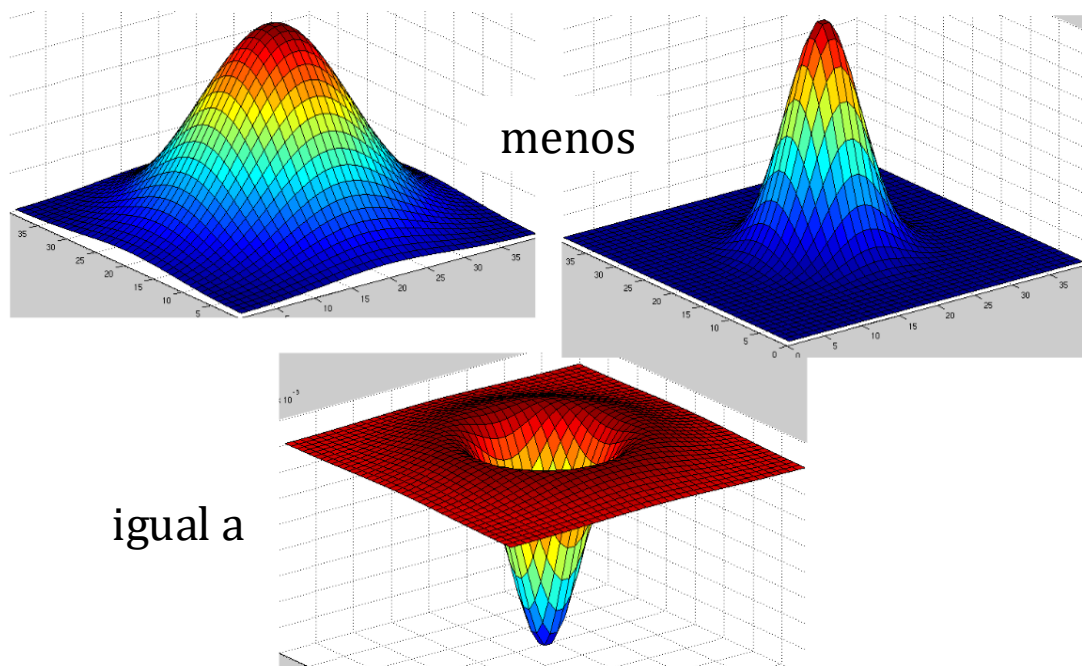
- Ojo, espacio de salida 3D ($X \times Y \times scale$).
- También se detectará la orientación característica.

Extracción de características

Paso 1: Detección de puntos clave (x,y,σ)



- ▶ Para forzar la invarianza a escala, se realiza un filtro DoG (*Difference of Gaussians*) a distintas escalas, y nos quedamos con mínimos locales del espacio de salida.
- ▶ DoG aproxima LoG (*Laplacian of Gaussian*); (el laplaciano es la suma de las segundas derivadas parciales, $D_{xx}+D_{yy}$):



0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

0	1	1	2	2	2	1	1	0
1	2	4	5	5	5	4	2	1
1	4	5	3	0	3	5	4	1
2	5	3	-12	-24	-12	3	5	2
2	5	0	-24	-40	-24	0	5	2
2	5	3	-12	-24	-12	3	5	2
1	4	5	3	0	3	5	4	1
1	2	4	5	5	5	4	2	1
0	1	1	2	2	2	1	1	0

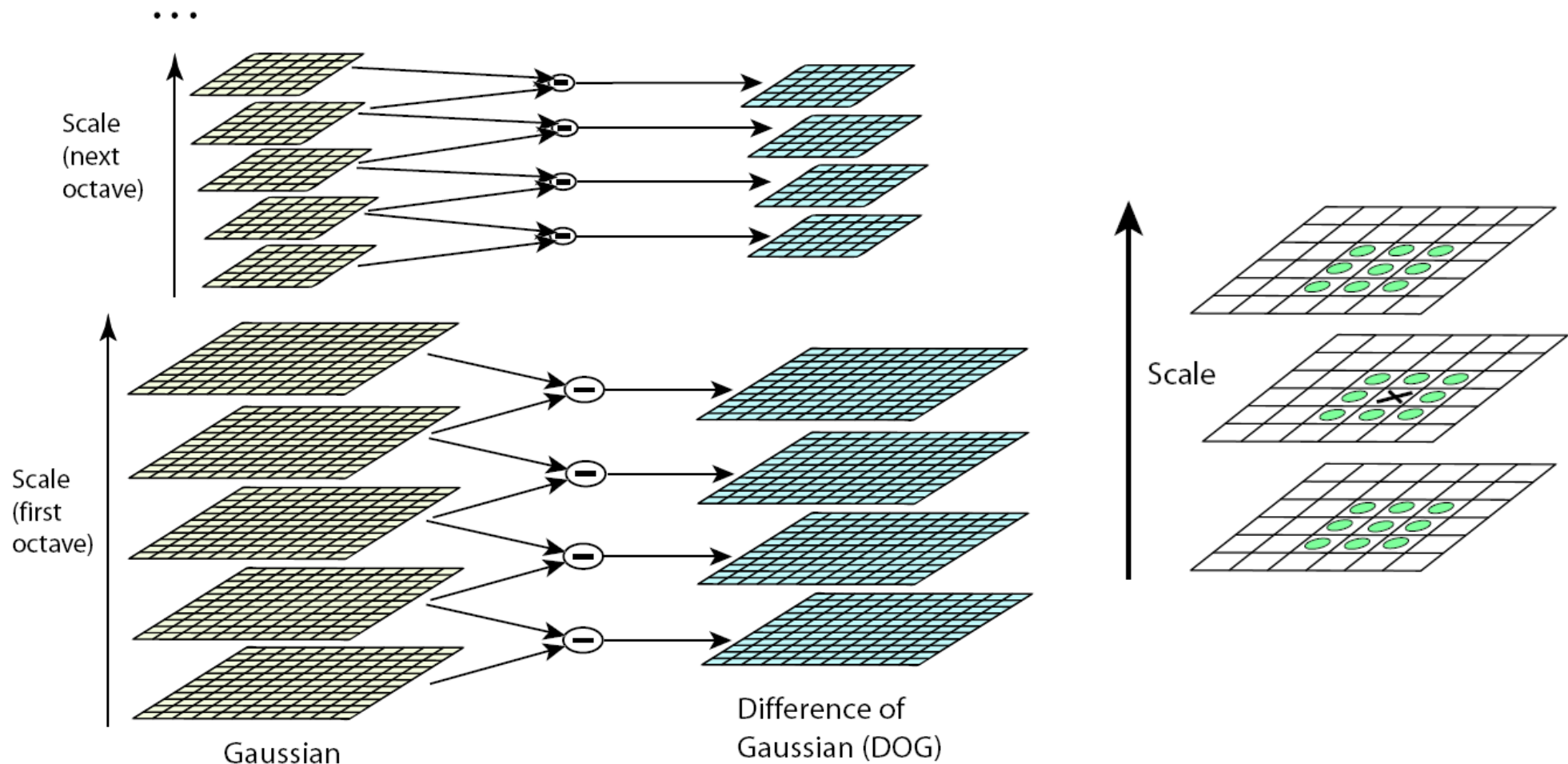
Es un simple **filtro lineal**,
cuyos coeficientes suman 0
(salvo posible redondeo)

Extracción de características

Paso 1: Detección de puntos clave (x,y,σ)



► Localización de máximos en espacio de escala:



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



Máster en
Inteligencia
Artificial
UMU

► Imagen original:



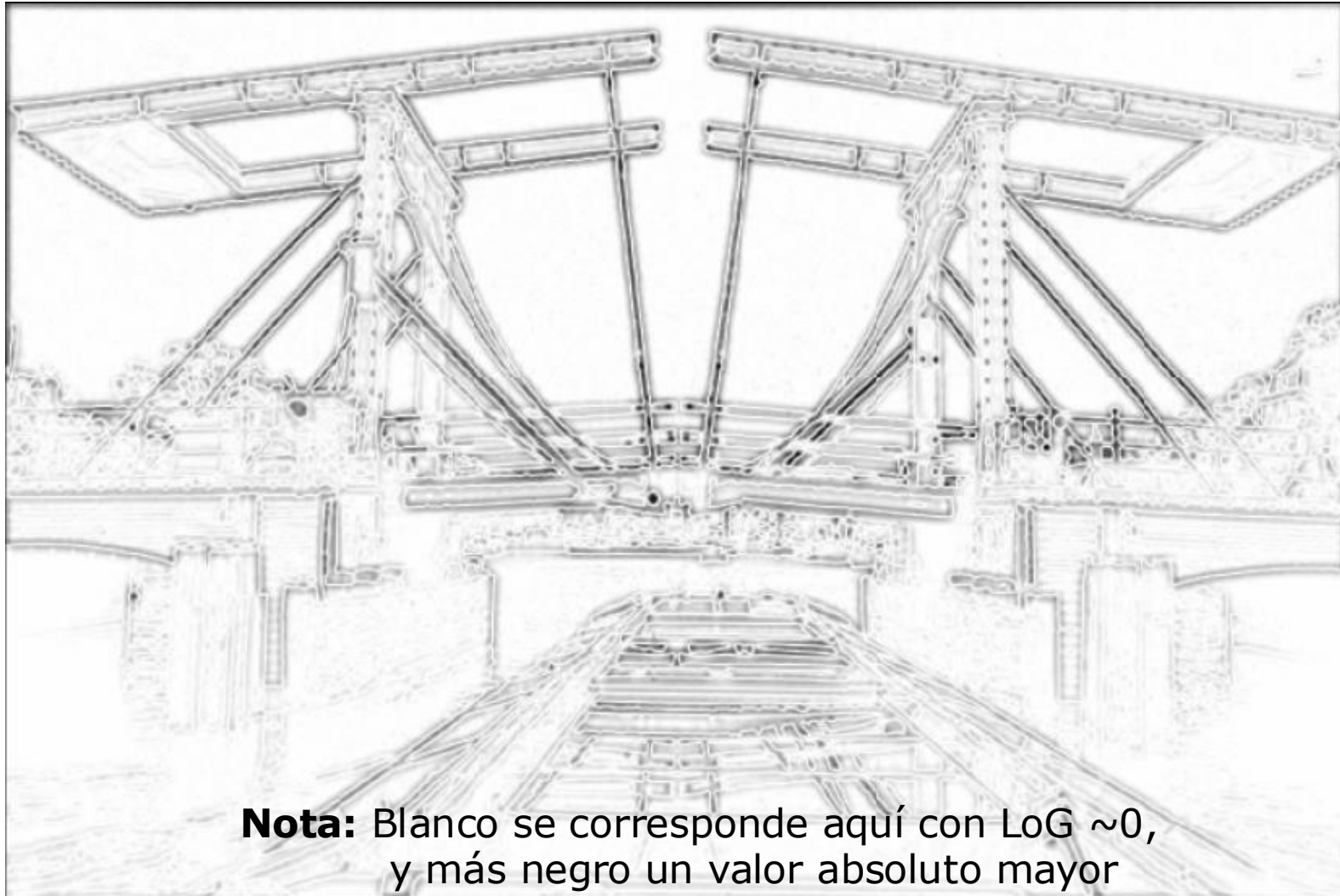
Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



Máster en
Inteligencia
Artificial
UMU

► Resultado LoG (con suavizados incrementales):



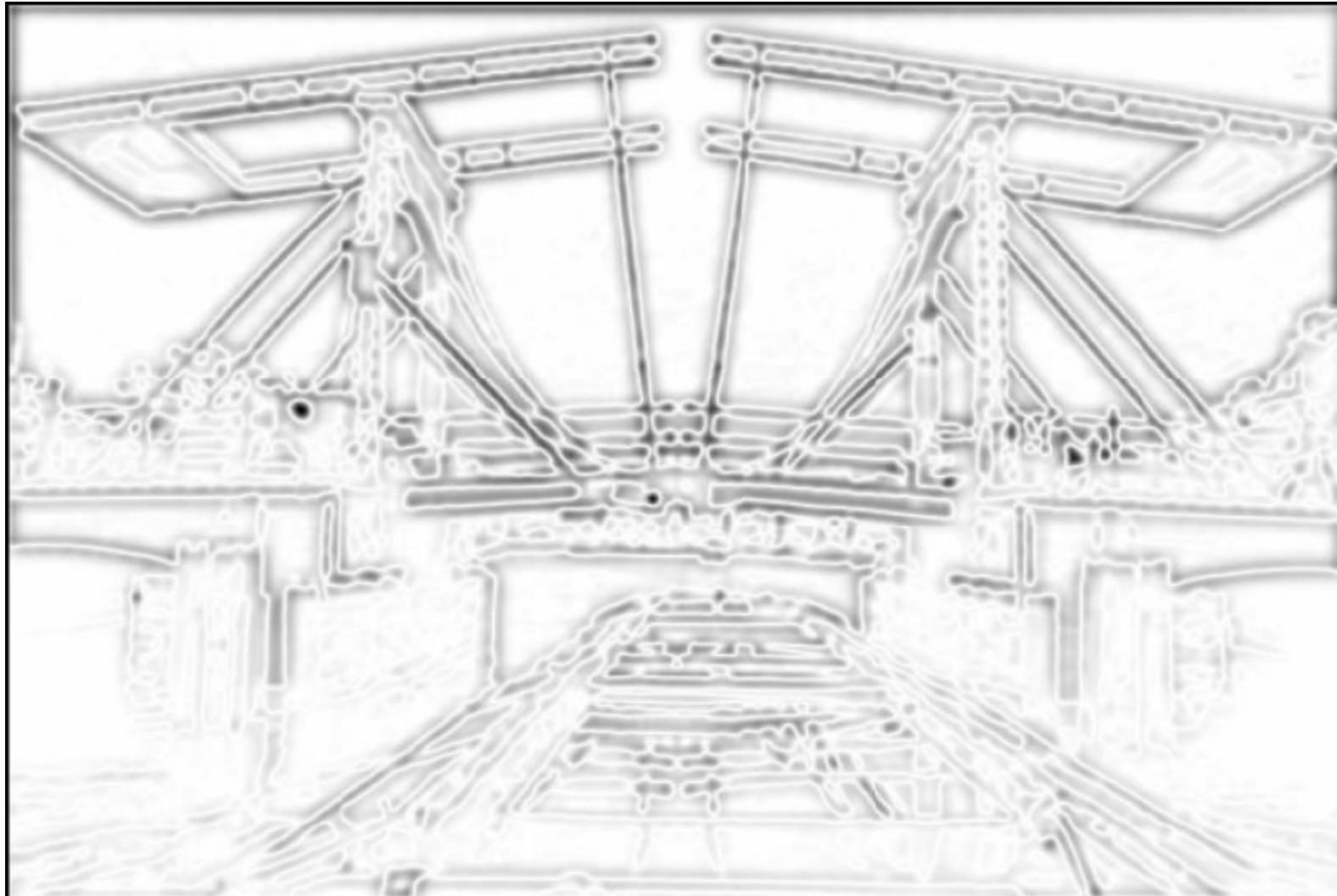
Nota: Blanco se corresponde aquí con $\text{LoG} \sim 0$,
y más negro un valor absoluto mayor

Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



► Resultado LoG (con suavizados incrementales):

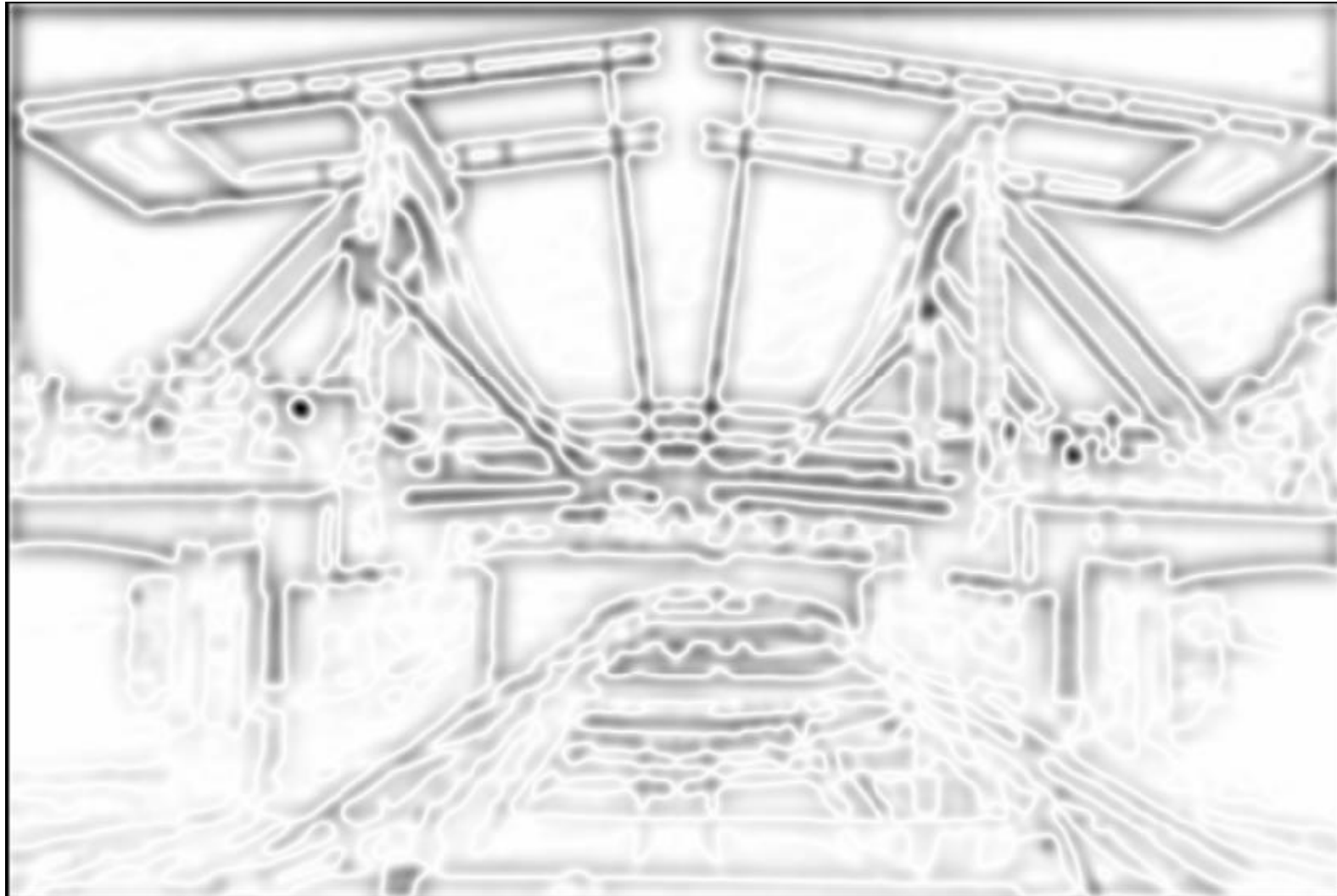


Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



► Resultado LoG (con suavizados incrementales):



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



Máster en
Inteligencia
Artificial
UMU

- Resultado LoG (con suavizados incrementales):



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



- Resultado LoG (con suavizados incrementales):



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



Máster en
Inteligencia
Artificial
UMU

- Resultado LoG (con suavizados incrementales):

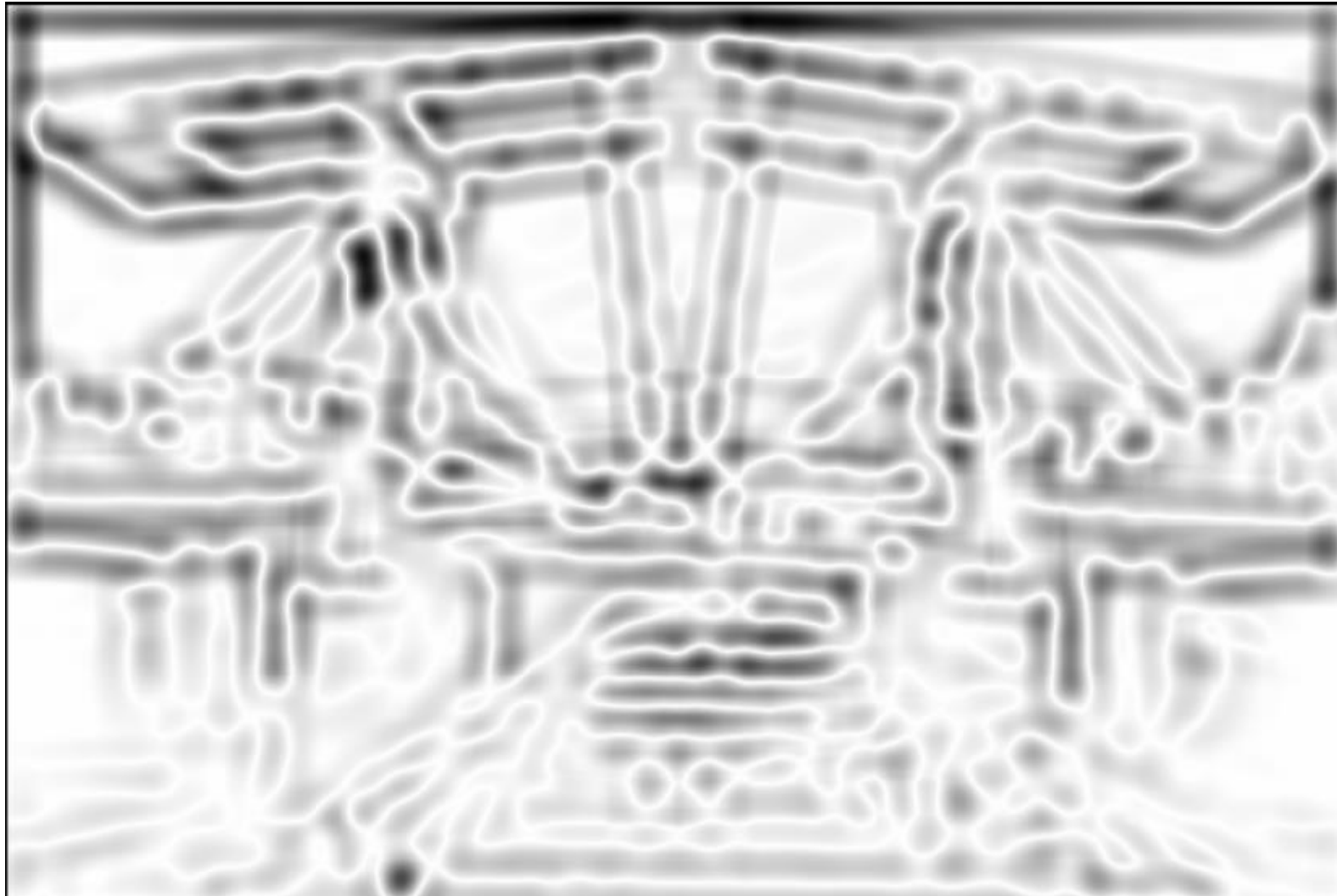


Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



- Resultado LoG (con suavizados incrementales):

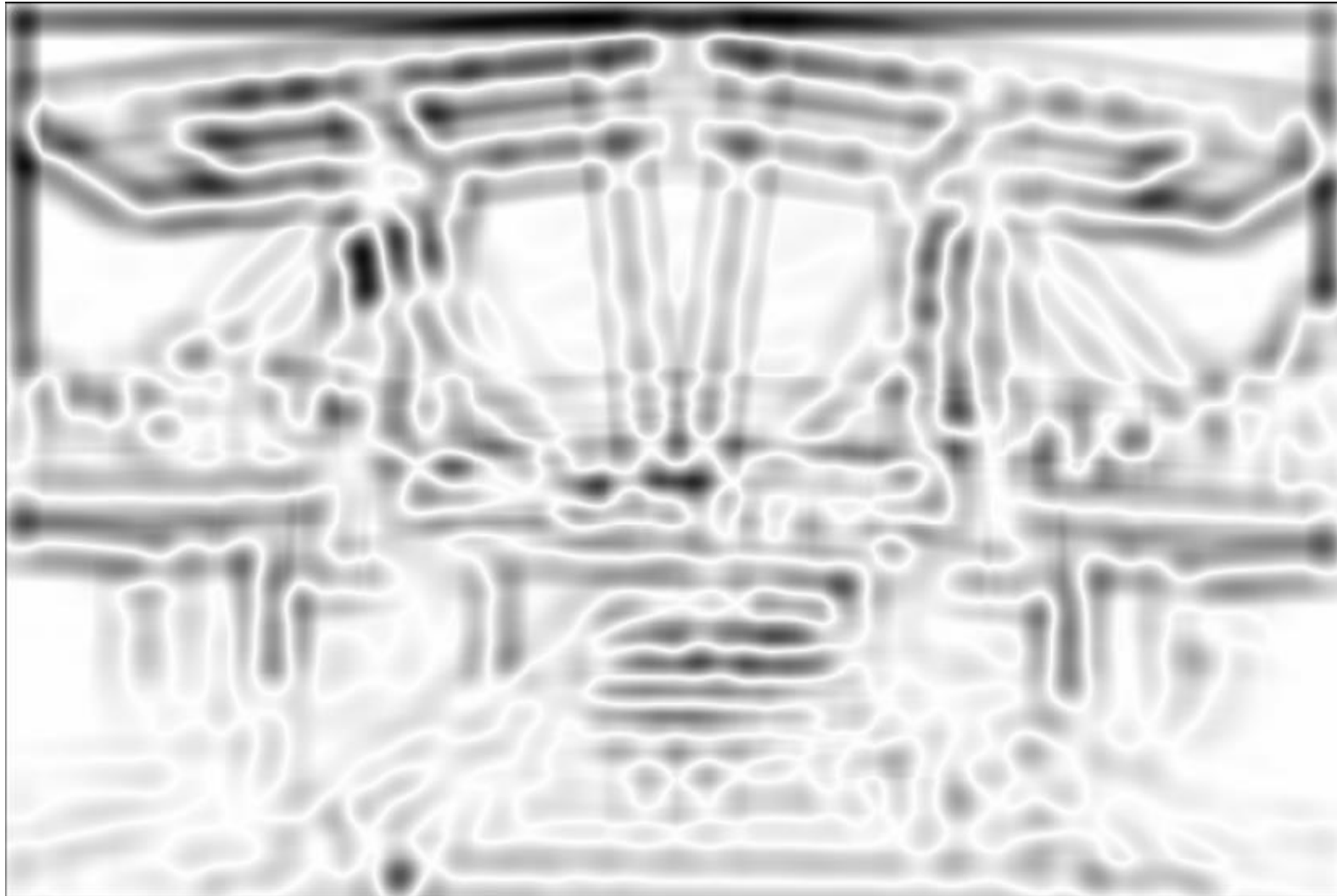


Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



- Resultado LoG (con suavizados incrementales):



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



- Resultado LoG (con suavizados incrementales):



Extracción de características

Espacio de escala de la LoG (*Laplacian of Gaussian*)



- Resultado LoG (con suavizados incrementales):



Extracción de características

Localización de máximos en espacio de escala LoG



$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

$$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y)$$

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

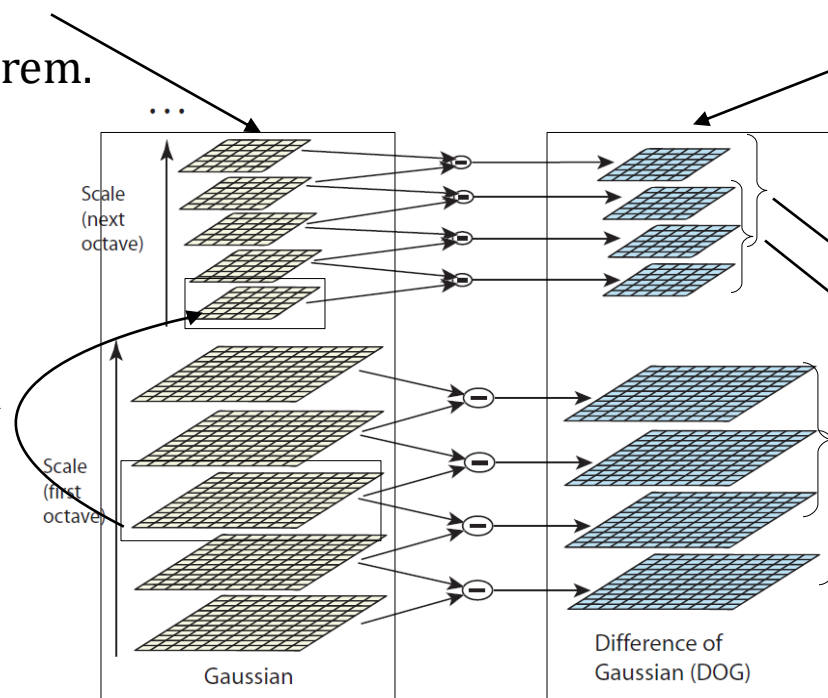
$$= L(x, y, k\sigma) - L(x, y, \sigma)$$

Filtros Gaussianos (increm.
 $k\sigma$ en cada paso)

Dif. de Gaussianas
(DoG)

Submuestreo
($W/2, H/2$)

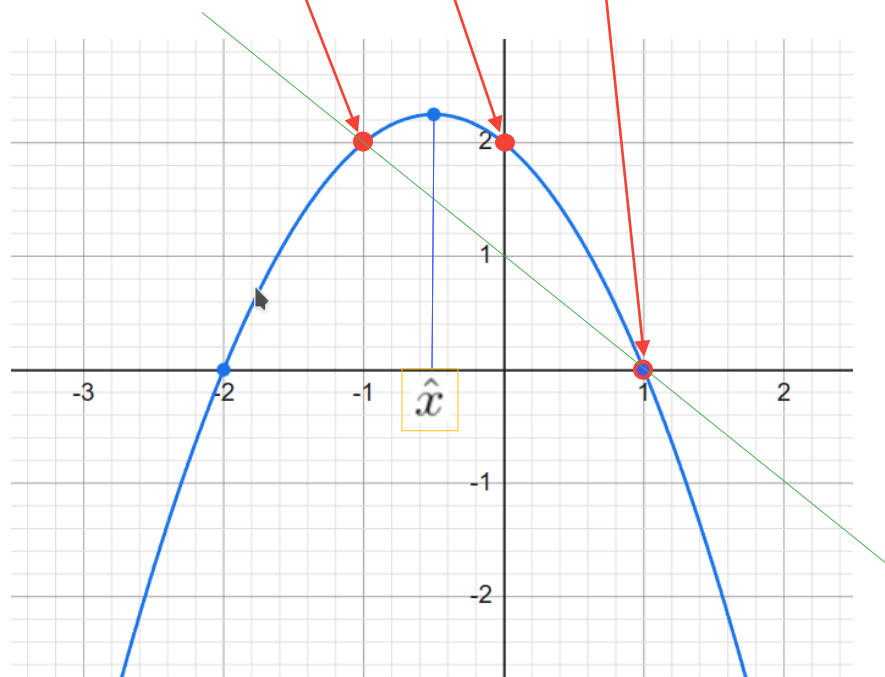
$s=2$ escalas
por octava



Detección de
máximos / mínimos
con plano sup. e inf.

En este ejemplo: $s=2 \rightarrow 2+3=5$ suavizados con $\sigma_0, \sigma_1=2^{1/2}\sigma_0, \sigma_2=2\sigma_0, \sigma_3=2^{3/2}\sigma_0$ y $\sigma_4=4\sigma_0$. Aquí se muestran dos octavas, y la imagen (antepenúltima) de cada octava es la que se submuestrearía (pixel sí, píxel no) a la mitad para empezar la siguiente.

- Interpolación (usando expansión de Taylor) del máximo de una parábola (1D) muestreada en 3 puntos alrededor del origen: $f(-1)$, $f(0)$, $f(+1)$:



Expansión de Taylor (2º orden):

$$\begin{aligned} f(x) &= f(0) + \underbrace{f'(0)}_{\text{green circle}}(x - 0) + \frac{1}{2}\underbrace{f''(0)}_{\text{blue circle}}(x - 0)^2 \\ &= 2 + (-1)x + \frac{1}{2}(-2)x^2 = 2 - x - x^2 \end{aligned}$$

Estimación 1ª y 2ª derivadas:

"Filtro Sobel": -1 0 +1

$$\underbrace{f'(0)}_{\text{green circle}} = \frac{f(+1) - f(-1)}{2} = \frac{0 - 2}{2} = -1$$

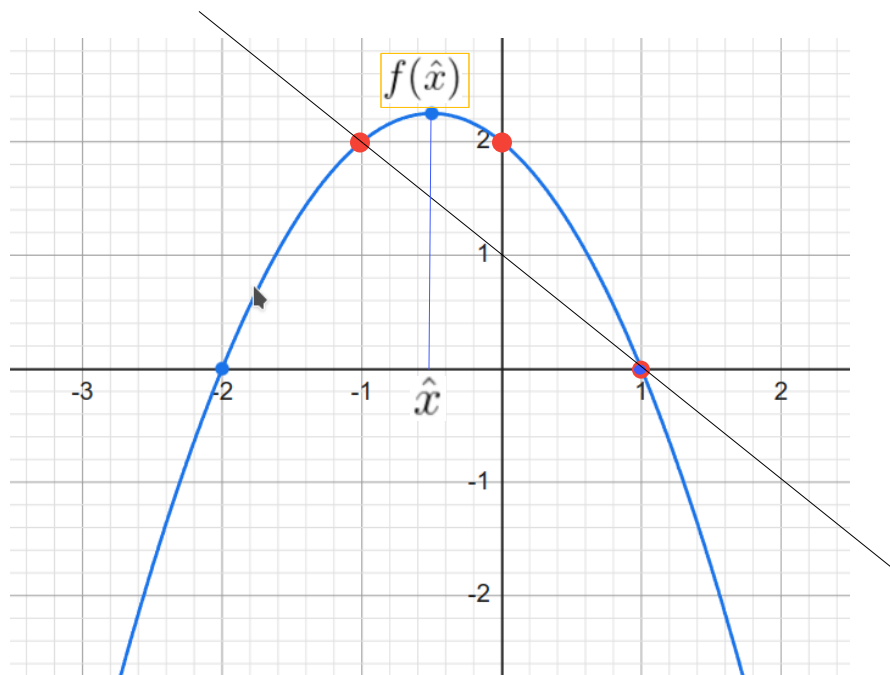
"Filtro LoG": +1 -2 +1

$$\begin{aligned} \underbrace{f''(0)}_{\text{blue circle}} &= f(-1) + f(1) - 2f(0) \\ &= 2 + 0 - 2 \cdot 2 = -2 \end{aligned}$$

Estimación posición del máximo ($\hat{x}$ donde la derivada se anula):

$$f'(\hat{x}) = 0 \Rightarrow f'(0) + f''(0)\hat{x} = 0 \Rightarrow \boxed{\hat{x} = -\frac{f'(0)}{f''(0)}} \rightarrow \hat{x} = -\frac{-1}{-2} = \boxed{-\frac{1}{2}}$$

- Estimación del valor de la función $f(\hat{x})$ en el óptimo $\hat{x}$:



$$f(\hat{x}) = f(0) + f'(0)\hat{x} + \frac{1}{2}f''(0)\hat{x}^2$$

$$= f(0) + f'(0)\hat{x} + \frac{1}{2}f''(0)\left(-\frac{f'(0)}{f''(0)}\right)^2$$

$$= f(0) + f'(0)\hat{x} + \frac{1}{2}f'(0)\frac{f'(0)}{f''(0)}$$

$$= f(0) + f'(0)\hat{x} + \frac{1}{2}f'(0)(-\hat{x})$$

$$= f(0) + \frac{1}{2}f'(0)\hat{x}$$

$$= 2 + \frac{1}{2}(-1)\left(-\frac{1}{2}\right) = \frac{9}{4}$$

► Precisión subpíxel y continuo de escalas en localización SIFT:

- Para cada máximo/mínimo, ajustar una función cuadrática 3D → análogo al desarrollo anterior para la parábola 1D, pero con expansión de Taylor 3D en lugar de 1D:

Diagram illustrating the 3D Taylor expansion of the function $D(\mathbf{x})$ around a point $\mathbf{x}$:

$$D(\mathbf{x}) = D + \frac{\partial D}{\partial \mathbf{x}}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

Annotations:

- $D(\mathbf{x})$: Función $\mathbb{R}^3 \rightarrow \mathbb{R}$
- D : Escalar
- $\frac{\partial D}{\partial \mathbf{x}}$: Vector 3D (gradiente func.)
- $\mathbf{x}$: Vector 3D $(\Delta x, \Delta y, \Delta \sigma)$
- $\frac{\partial^2 D}{\partial \mathbf{x}^2}$: Matriz 3x3 (hessiano func.)

D and its derivatives are estimated directly at each initial point of the grid (x, y, σ) , and here $\mathbf{x} = (\Delta x, \Delta y, \Delta \sigma)$

- Computar la posición exacta del máximo/mínimo con precisión subpíxel derivando e igualando a cero:

$$\hat{\mathbf{x}} = -\frac{\partial^2 D}{\partial \mathbf{x}^2}^{-1} \frac{\partial D}{\partial \mathbf{x}} \quad \xleftrightarrow{3D \leftrightarrow 1D} \quad \boxed{\hat{x} = -\frac{f'(0)}{f''(0)}}$$

- El valor de D en dicho extremo se calcula sustituyendo la solución en la primera ecuación:

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2} \frac{\partial D}{\partial \mathbf{x}}^T \hat{\mathbf{x}} \quad \xleftrightarrow{3D \leftrightarrow 1D} \quad \boxed{f(0) + \frac{1}{2} f'(0) \hat{x}}$$

Recordar enlaces sobre aproximaciones de Taylor:

[\(1D\)](#) [\(2D\)](#)

Extracción de características

Paso 3: Eliminar puntos inestables



- Eliminar puntos con un valor absoluto del máximo/mínimo por debajo de un umbral:

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{x}} \hat{\mathbf{x}} \leftarrow \text{Si no supera un umbral } |D(\hat{\mathbf{x}})| > 3\% \text{ del valor máximo de la imagen, se descarta por inestable.}$$

- Computar ratio de valores propios del Hessiano (recordar, al estilo del detector de Harris, pero con segunda derivada en lugar de orientación del gradiente), y eliminar puntos con ratio por debajo de un umbral:

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

Submatriz 2x2 del hessiano anterior

Para calcular el ratio r entre valores propios α y β de $\mathbf{H}$ no hace falta la descomposición explícita, bastan la traza y el determinante (bastante más rápido):

$$\text{Tr}(\mathbf{H}) = D_{xx} + D_{yy} = \alpha + \beta,$$

$$\text{Det}(\mathbf{H}) = D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta.$$

$$\frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r + 1)^2}{r}$$

Despejamos r (las dos soluciones son una la inversa de la otra)

Extracción de características

Ejemplos de detección



- Salida del detector (incluye posición, escala y orientaciones dominantes; más sobre esto último a continuación).



► Forzar invarianza a la orientación:

- Computar orientación mediante **dirección dominante del gradiente** en la correspondiente imagen suavizada (histograma 36 posiciones, una por cada 10°).
- Magnitud m y orientación θ del gradiente en (x,y) (*ídem Canny*):

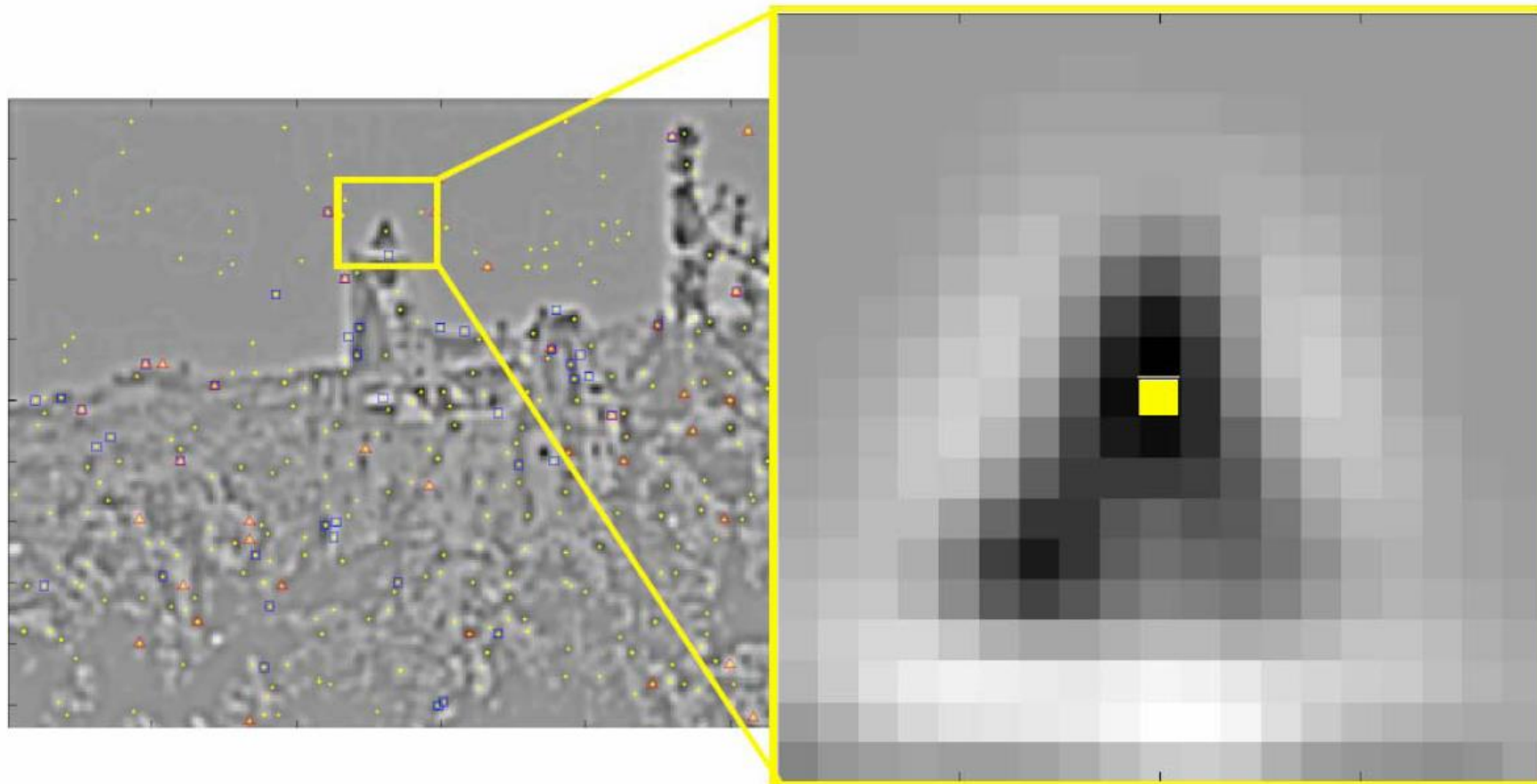
$$m(x, y) = \sqrt{(L(x + 1, y) - L(x - 1, y))^2 + (L(x, y + 1) - L(x, y - 1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y + 1) - L(x, y - 1)) / (L(x + 1, y) - L(x - 1, y)))$$

- Rotar el parche de imagen para que **esa orientación apunte hacia arriba** (en realidad, de nuevo interpolación cuadrática, para detectar el pico del histograma algo mejor).

Extracción de características

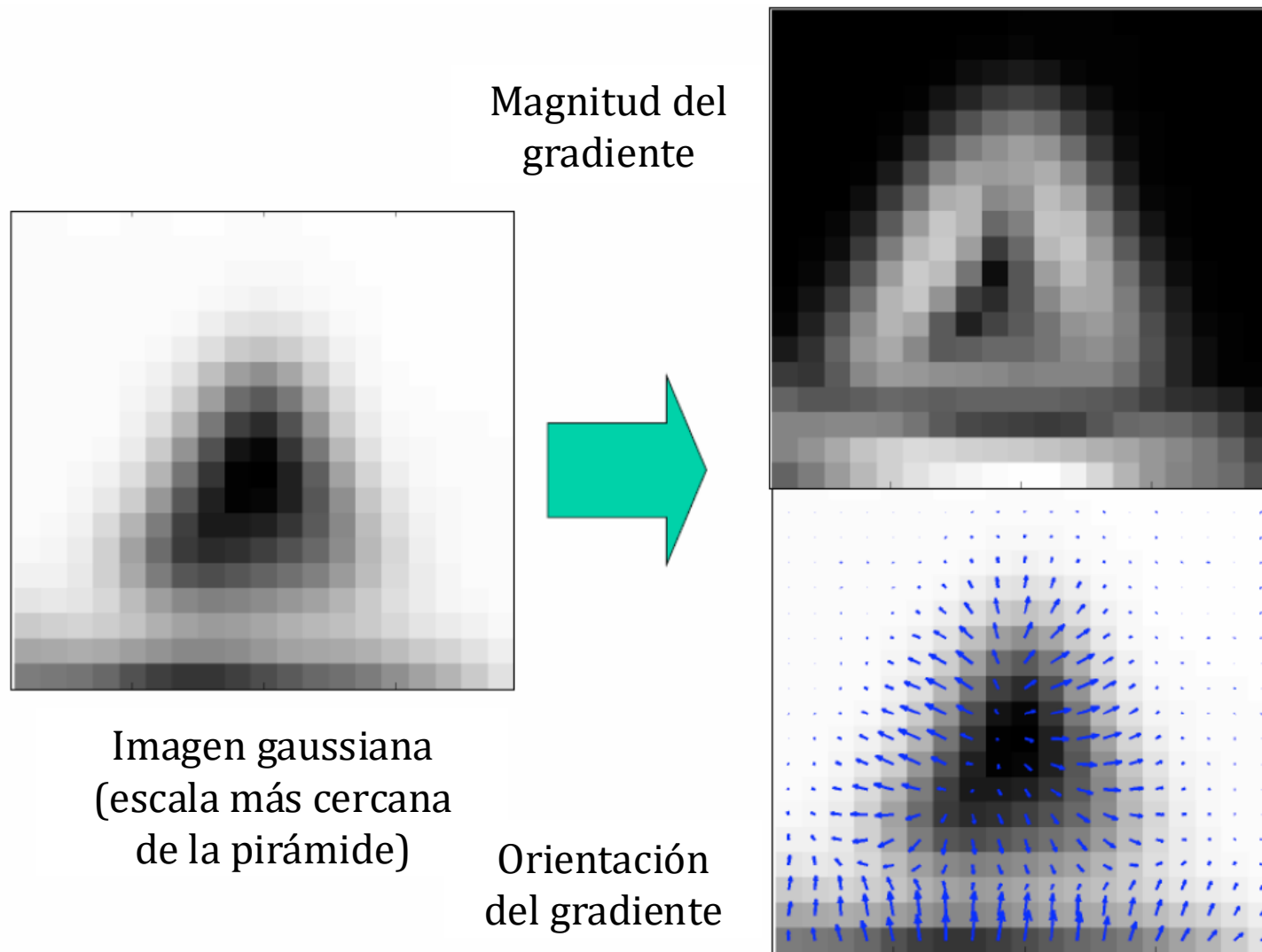
Asignación de orientación a cada *feature* (II)



Localización del *keypoint* = posición extremo
Escala del *keypoint* = escala de la imagen DoG

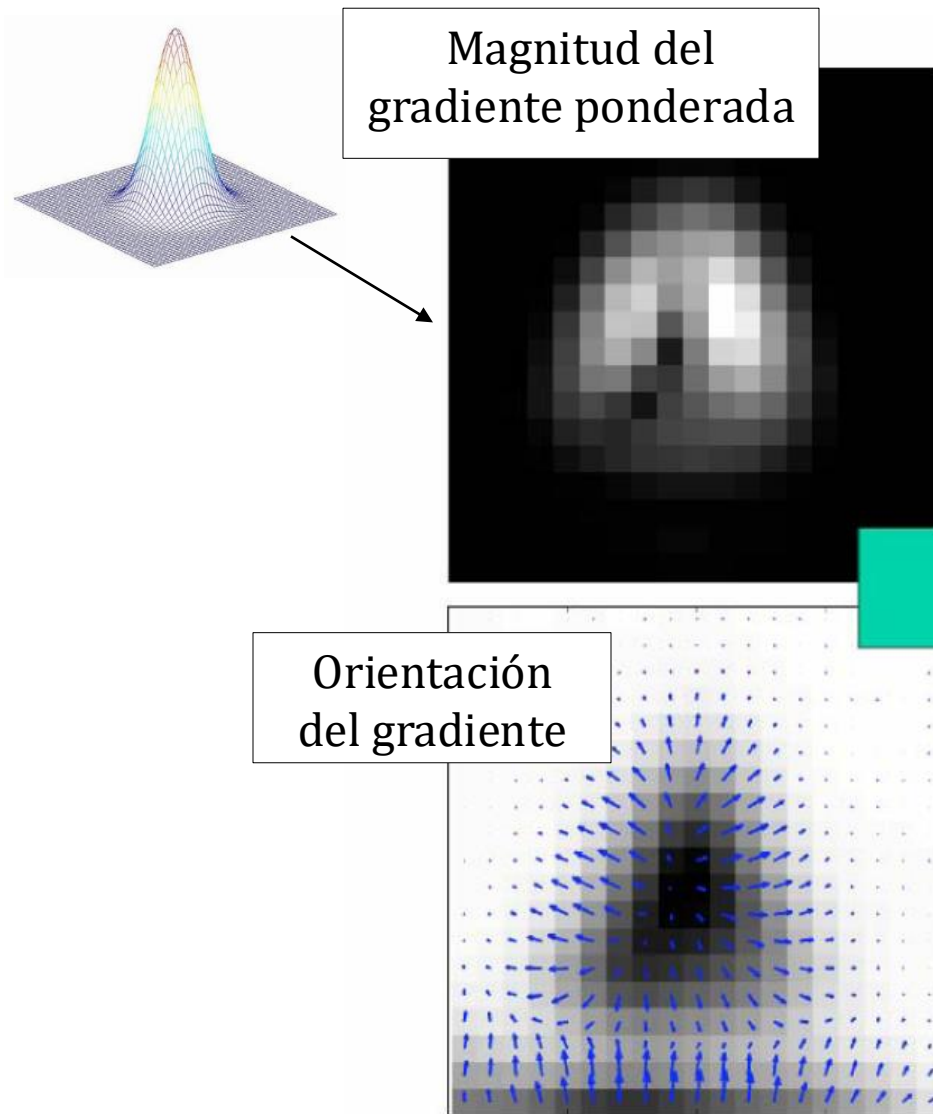
Extracción de características

Asignación de orientación a cada *feature* (III)

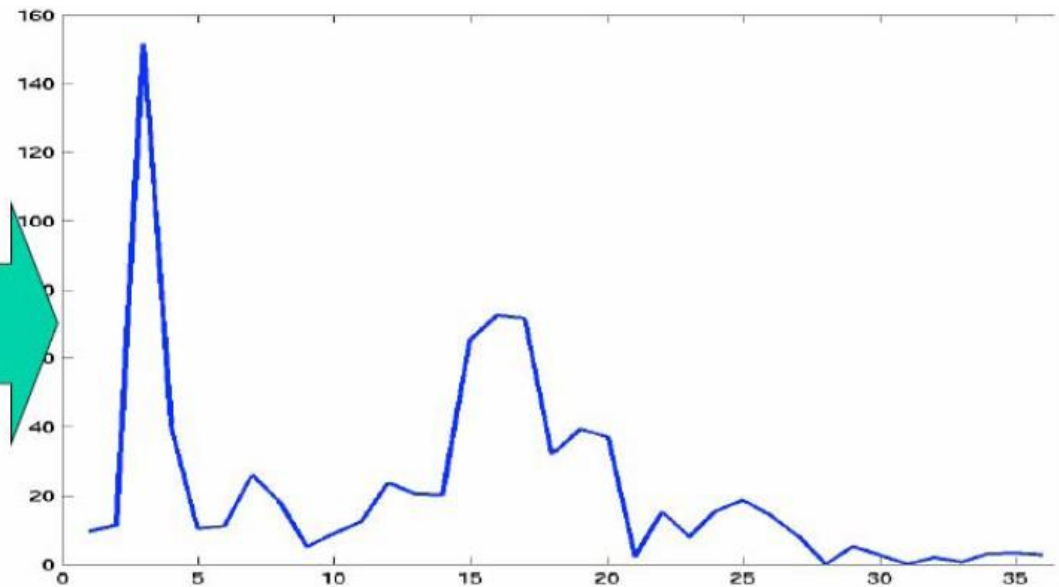


Extracción de características

Asignación de orientación a cada *feature* (IV)



Histograma de orientaciones ponderado
- Cada *bin* contiene suma de magnitudes del gradiente ponderadas para ángulos que caen en dicho *bin*.

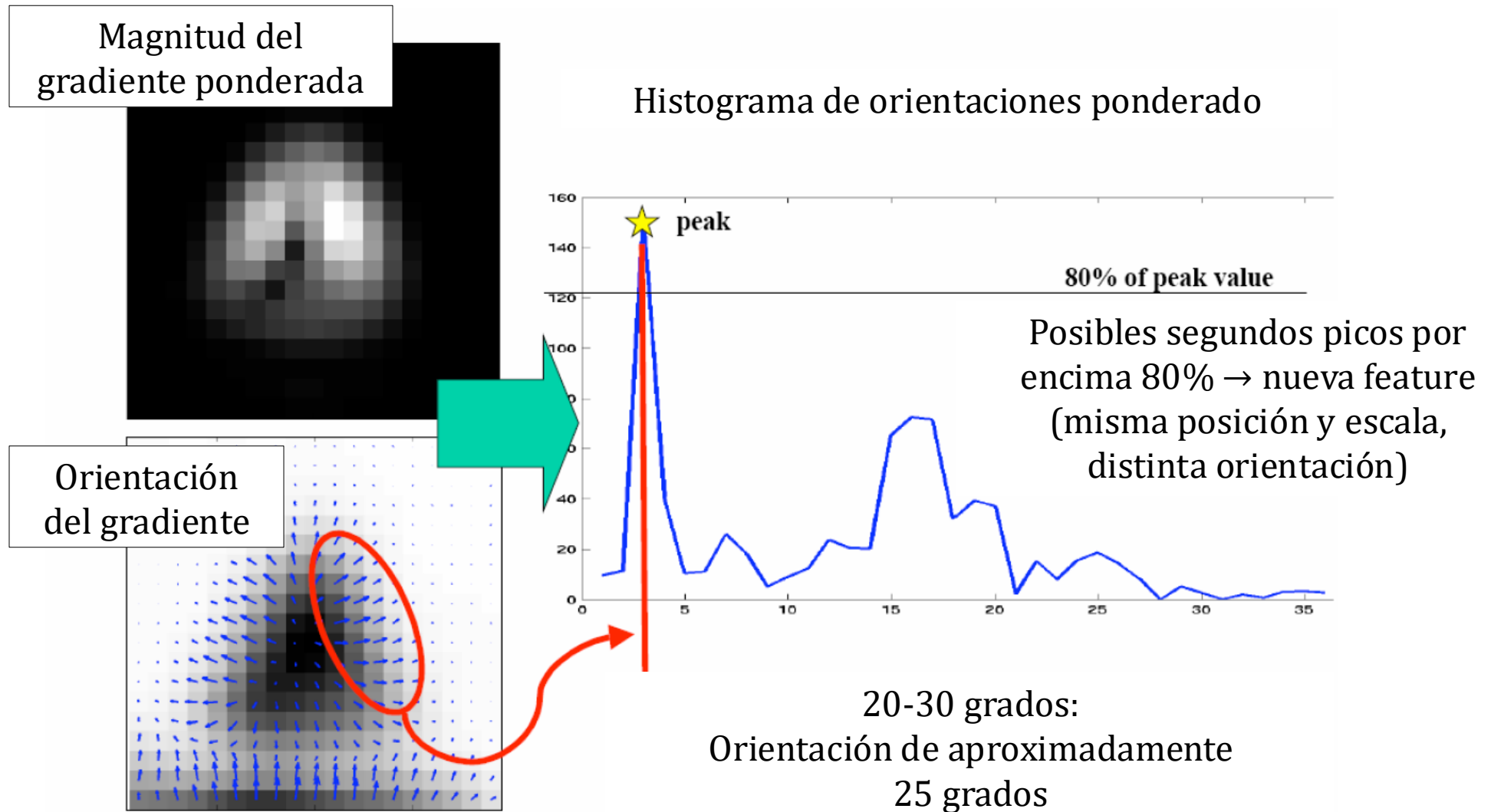


36 bins (10° cada uno)

- $0 \leq \text{ang} < 10$: *bin 1*
- $10 \leq \text{ang} < 20$: *bin 2*
- ...
- $350 \leq \text{ang} < 360$: *bin 36*

Extracción de características

Asignación de orientación a cada *feature* (V)

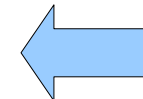
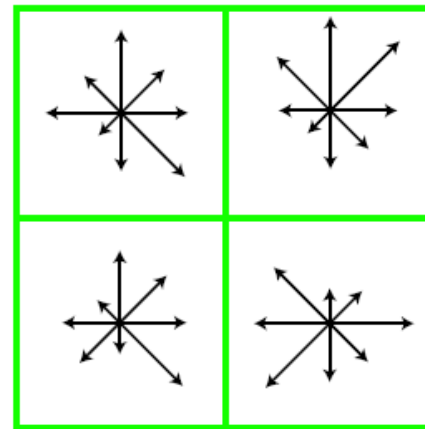
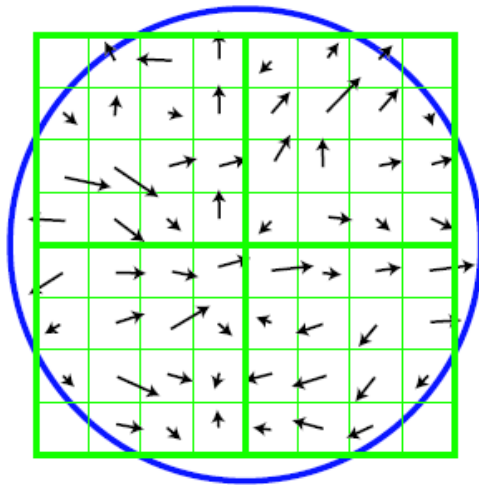
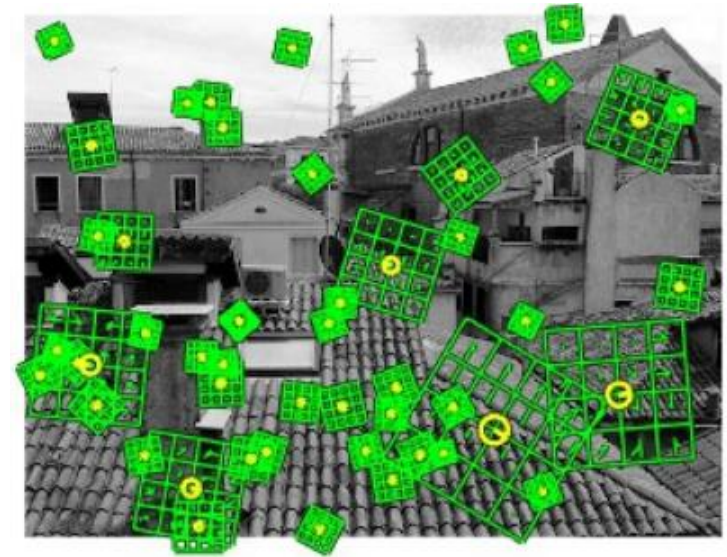


Extracción de características

Caracterización de los puntos (*descriptors*) (I)



- ▶ Computar histograma del gradiente en la región de la *feature*, dividida a su vez en 4x4 regiones (celdas cuadradas de píxeles escaladas y rotadas).
- ▶ El resultado es un vector de características con 128 valores (16 campos, discretización de 8 orientaciones de gradiente en cada campo).



Ejemplo simplificado:
si fuesen 2x2 regiones
de 4x4 píxeles cada una,
saldría como descriptor
un vector de dimensión 32

Gradientes de la imagen

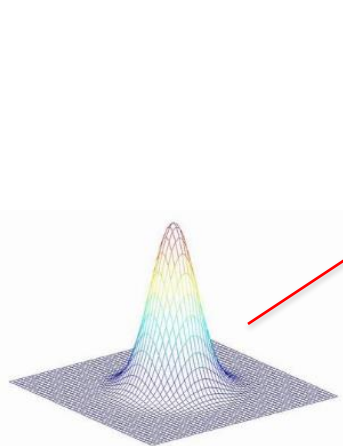
Descriptor del *keypoint*

Extracción de características

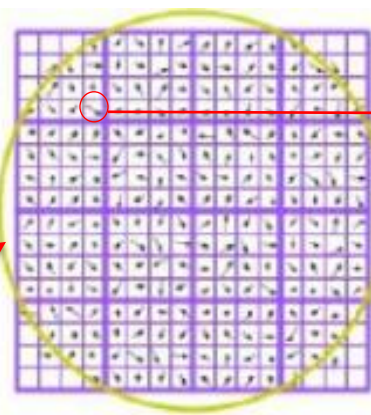
Caracterización de los puntos (*descriptors*) (II)



- ▶ Cada uno de los $16 \times 16 = 256$ gradientes del parche acumula su magnitud en ciertos *bins* de los histogramas de orientación de las 4×4 celdas.
- ▶ La acumulación se "suaviza": cuanto más se acerque una posición de gradiente a los bordes de la celda, más peso se "cederá" a los *bins* de orientación correspondientes en las correspondientes celdas adyacentes. Ídem para la orientación:
 - E.d., repartimos la magnitud en $2 \times 2 \times 2$ "votos": las dos celdas más cercanas a la posición del píxel en X, en Y, y los 2 *bins* de las orientaciones discretas más cercanas a la orientación original del gradiente en ese píxel, usando pesos d y $1-d$, siendo d la distancia normalizada en unidades de espaciado de *bins* (estilo interpolación trilineal, pero repartiendo peso, en lugar de interpolando).
 - También ayuda el "pesado" previo de los gradientes con una ventana gaussiana (mayor importancia cerca del centro del parche).

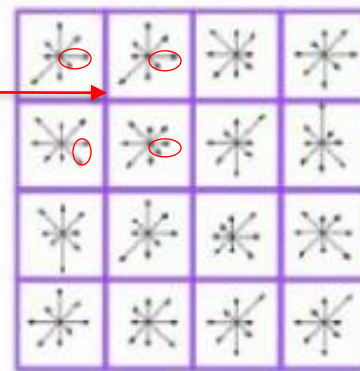


Suavizado gaussiano

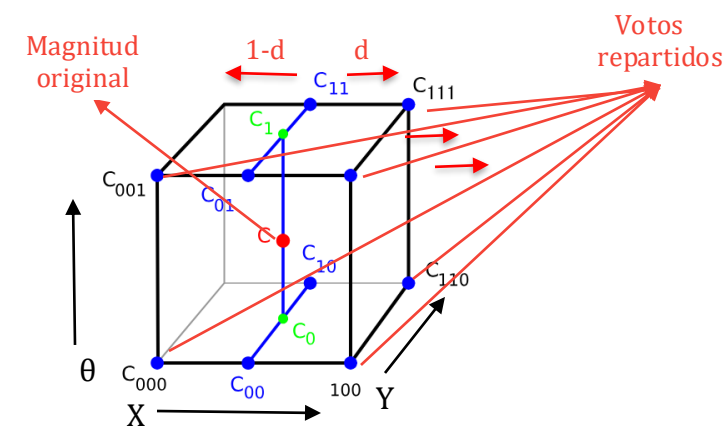


Gradientes (rotados)

$2 \times 2 \times 2 = 8$ votos



Descriptor resultante



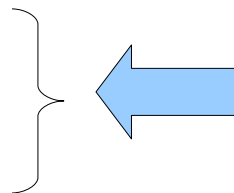
Interpolación trilineal



- ▶ **Normalizar vector a longitud** uno para obtener invarianza (afín, e.d, *bias* y *scale*) a cambios *lineales* de iluminación.
- ▶ Finalmente, “recortar” valores muy positivos del vector, y volver a normalizar, para obtener cierta invarianza a cambios no lineales de iluminación (saturación, etc.)
 - Propuesta Lowe → truncar componentes a ≤ 0.2 , y volver a normalizar.
- ▶ Potenciamos así la influencia de la orientación del gradiente, más que de su magnitud.

- ▶ Se realizará partiendo de un **paso previo de *matching* individual** (*nearest neighbour*) de cada *keypoint* contra toda una base de datos de *keypoints* extraídas de las imágenes de interés (patrones a buscar, guardadas y preprocesadas).
- ▶ Pero muchos *keypoints* ni siquiera estarán en la base de datos (serán del fondo, otros objetos, etc.). Además, el *matching* puede fallar con relativa facilidad.
- ▶ Necesario un **esquema robusto** para estimar los *matchings* correctos →

- RANSAC, ó
- Transformada de Hough



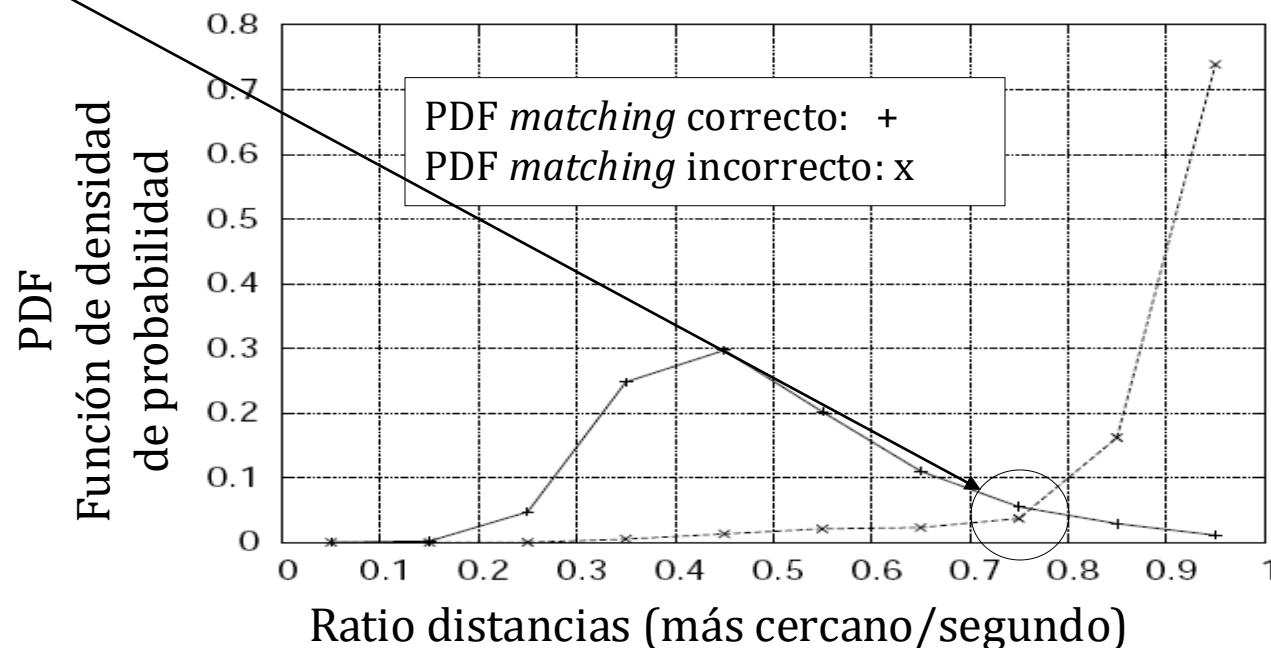
Aplicadas en la
**estimación de la
homografía**

Extracción de características

Reconocimiento de objetos con SIFT (II)



- ▶ Muchas veces el *nearest neighbour* (NN) no se corresponderá con el *keypoint* correcto de la base de datos.
- ▶ Así que habrá que rechazar *a priori* algunos *matchings*. Pero un umbral absoluto no funciona bien...
- ▶ ...así que para aceptarlo, el 2º NN deberá estar, como poco, a un 20% de distancia mayor que el 1º:



- ▶ También es interesante la idea del ***matching mutuo*** (para prefiltrar posibles *matchings* espurios):
 - Localizar la *feature* Y de la imagen B cuyo descriptor es el más cercano al descriptor de una *feature* X dada en la imagen A.
 - Comprobar entonces si, en sentido contrario, también hay *matching*:
 - (Es decir, si la *feature* de la imagen A cuyo descriptor es el más cercano al descriptor de Y es X).
 - Si no es así → eliminar posible *matching*.

Extracción de características

Reconocimiento de objetos con SIFT (IV)



- Si se tienen **tres** o más *matchings* correctos (*inliers*), se puede calcular la transformación **afín** modelo-imagen:

Posición en la imagen $\rightarrow \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$

Posición en el modelo $\rightarrow$

Transformación afín y traslación a estimar

Cada *match* contribuye con dos ecuaciones...

$$\begin{bmatrix} x & y & 0 & 0 & 1 & 0 \\ 0 & 0 & x & y & 0 & 1 \\ \vdots & & & & & \\ \vdots & & & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_x \\ t_y \end{bmatrix} = \begin{bmatrix} u \\ v \\ \vdots \end{bmatrix}$$

$N \times 6$ matriz $\rightarrow \mathbf{A} \mathbf{x} = \mathbf{b}$ $\leftarrow N \times 1$ vector

Resolvemos el sistema por mínimos cuadrados, usando al menos 3 *matchings*

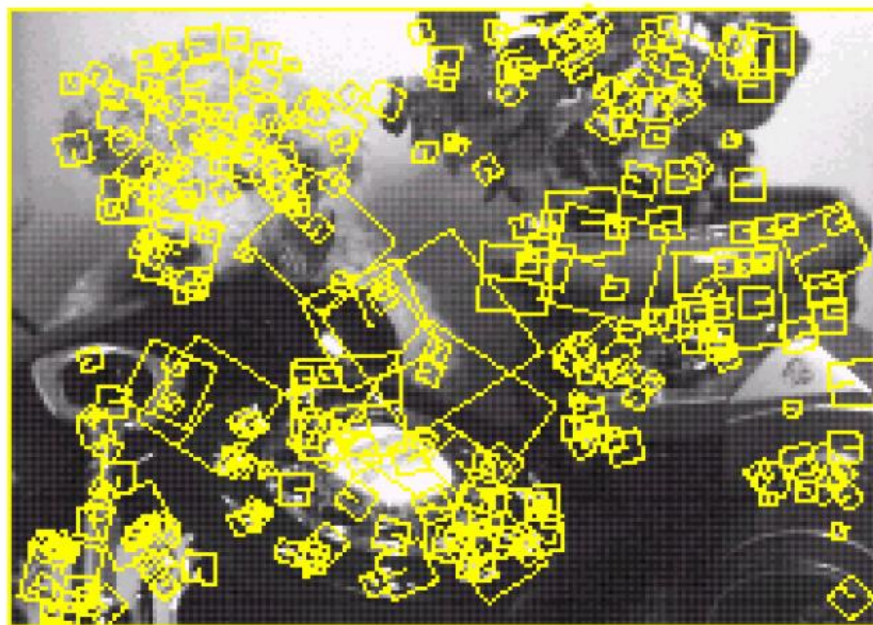
$$\mathbf{x} = [\mathbf{A}^T \mathbf{A}]^{-1} \mathbf{A}^T \mathbf{b}$$

Extracción de características

Ejemplos (I)



- Ejemplo de estabilidad en diferentes condiciones de luz:



273 *matchings* verificados

Extracción de características

Ejemplos (II)



► Ejemplo de localización (incluso con ocultación):



Localizar modelos como éstos...



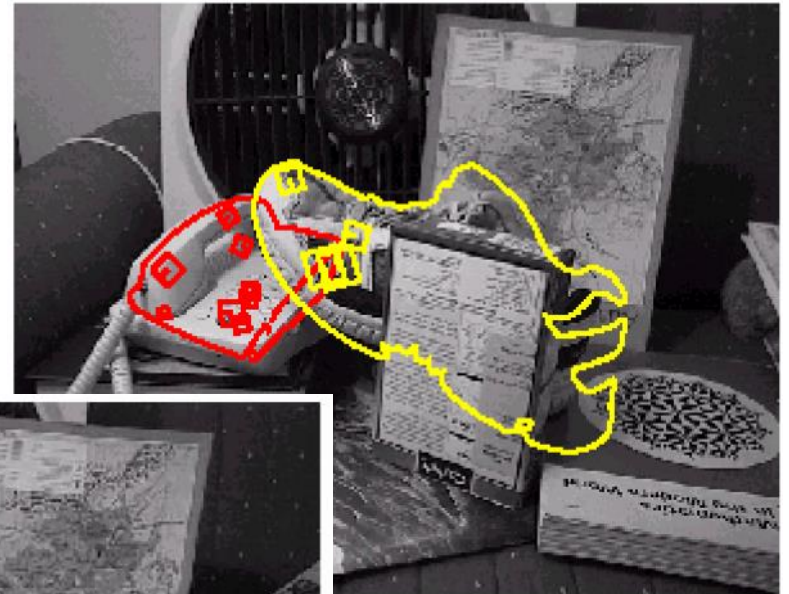
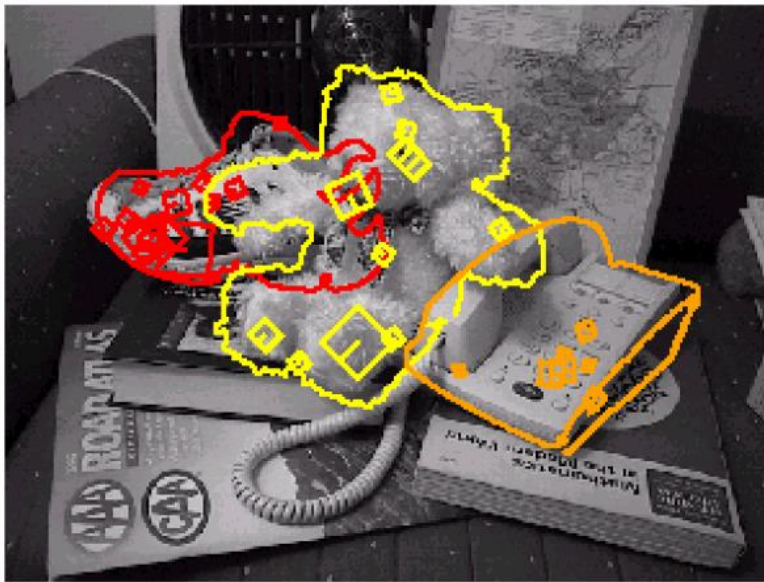
...en imágenes con *clutter* como ésta.

Extracción de características

Ejemplos (III)



► Ejemplo de localización (incluso con ocultación):



Extracción de características

Ejemplos (IV)



Máster en
Inteligencia
Artificial
UMU

► Resultados ejercicio práctico de reconocimiento objetos planares:



Extracción de características

Ejemplos (V)



- Resultados ejercicio práctico de reconocimiento objetos planares:



► **Localidad** en la detección:

- No involucra más que cálculos en el entorno de la característica.
- Ello implica cierta **eficiencia** (aunque, ojo, el cálculo a todas las escalas lo hace más lento).
- La localidad también implica resistencia a la oclusión y al ruido de fondo (*background clutter*).

► Capacidad **discriminante** de los descriptores:

- *Matching* individual de *features* contra grandes BBDDs.

► **Cantidad** de detecciones:

- Típicamente ~miles *features* estables, a distintas escalas, en imágenes de tamaño típicas (~1 MPixel).

- ▶ Algunos notebooks de Python+OpenCV:
 - Bordes: gradiente, operador de Canny, difusión de color: [↵](#) [↵](#)
 - Covarianza, direcciones principales: [↵](#)
- ▶ Tutorial Python+OpenCV (secciones relevantes):
 - Features (discusión general): [↵](#)
 - Operador de Canny: [↵](#)
 - Detector de Harris: [↵](#)
 - Variante Shi-Tomasi: [↵](#)
 - Speeded Up Robust Features (SURF): [↵](#)
 - Scale Invariant Feature Transform: SIFT: [↵](#)
- ▶ *Paper* SIFT:
 - “*Distinctive Image Features from Scale-Invariant Keypoints*”. D. Lowe, 2004.